NN&DL

Convolutional Neural Networks

S. Coniglio

Dipartimento di Scienze Economiche, UniBg

Sem. II

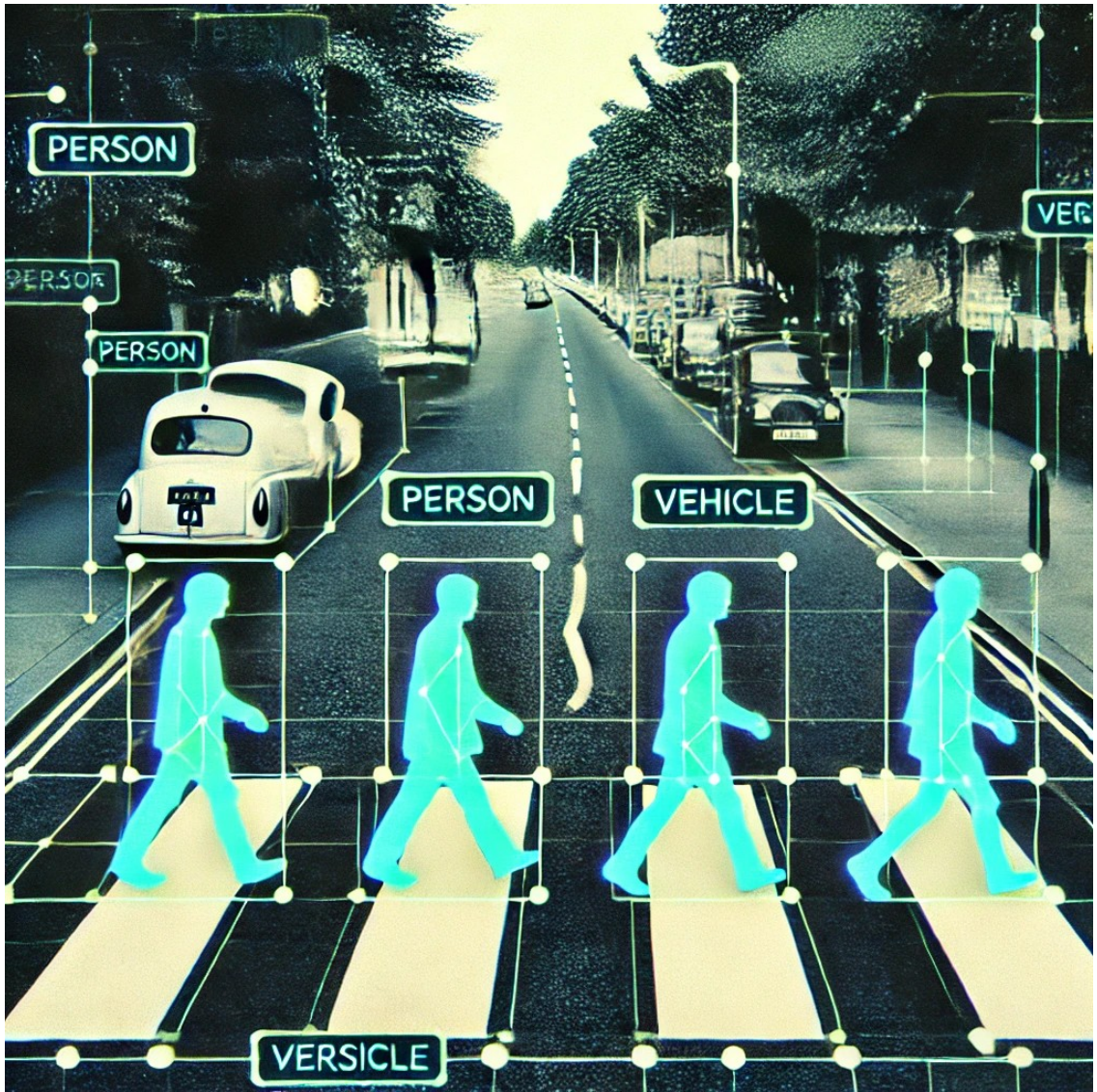
Summary

1. Computer vision
2. Why CNNs and not MLPs?
3. Historical perspective
4. 1D convolution
5. 2D convolution
6. Downsampling and 1×1 convolution
7. Training CNNs to achieve invariance
8. Closing

1. Computer vision

Computer vision

Simply put, **Computer Vision** (CV) is the branch of AI dedicated to the analysis and interpretation of image data.



"Solve vision"

In 1966, Marvin Minsky and Seymour Papert at MIT famously launched the **Summer Vision Project**: a summer internship in which a group of undergraduates was tasked with "solving vision": connecting a camera to a computer and getting the machine to *describe what it saw*.

Needless to say, not much resulted from this—this is in line with the early enthusiams in AI reserach which, as we know, failed quite often to deliver.

Indeed, sixty years later, computer vision (CV) is still very much an open problem, although huge progress has been made (especially in the last fifteen years, thanks to deep learning).

CV is one of the two largest subfields of AI (the other one being natural language processing — NLP). Let's see a few examples of CV applications.

Prototypical CV applications

- **image classification**: label the image based on its content
- **image captioning**: generate a textual description of the image
- **object detection**: recognize and localize objects within the image (crucial in autonomous driving)
- **image segmentation**: break the image down into regions, classifying individual pixels (e.g., different anatomical structures in a medical scan)
- **inpainting**: reconstructing missing parts of the image
- **style transfer**: transform an image to mimic the artistic style of another
- **upscaling**: enhance the image resolution by generating extra pixels (the infinite zoom of crime shows in the 90s!)
- **depth prediction/scene reconstruction**: infer three-dimensional information from a two-dimensional image
- **image synthesis**: generate images based on textual descriptions—this is a bit of a different task, though...

Image classification

Image classification is the prototypical CV problem: we know how to solve it only by using neural networks and not with symbolic (i.e., rule/algorithm-based) techniques.

Notably, humans do not execute a precise algorithm when distinguishing a photo of a cat from that of a dog—they do it almost instinctively.

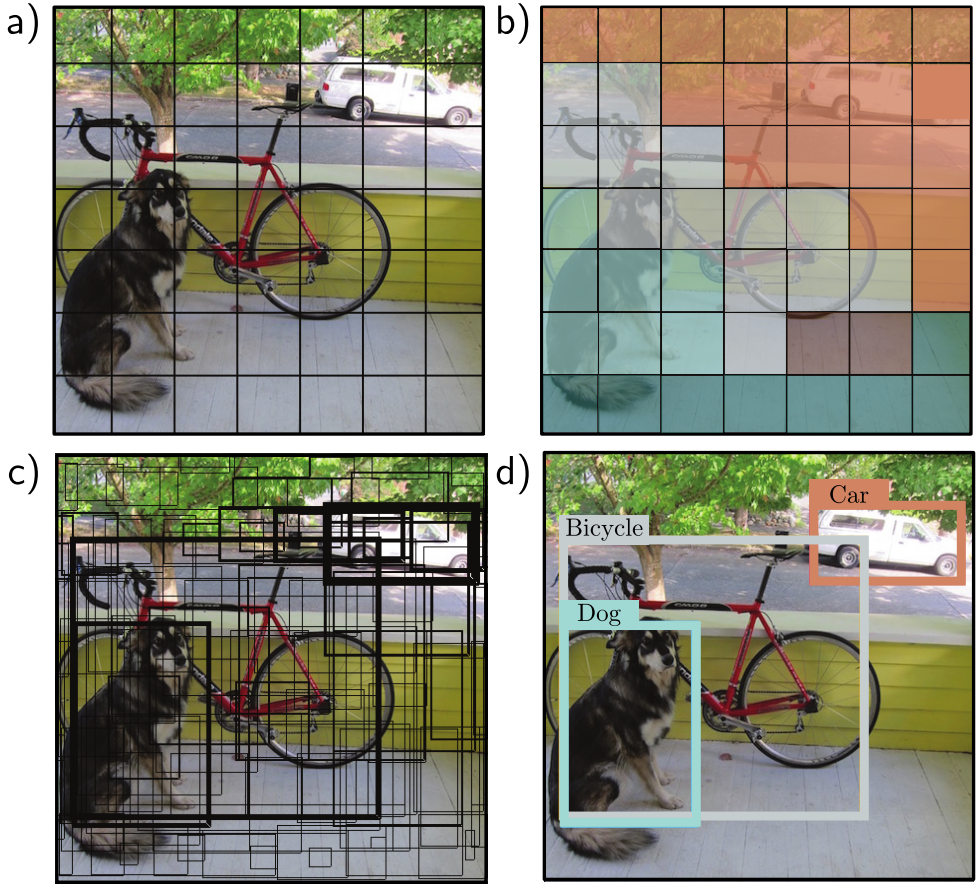
Doing so require a form of *tacit knowledge* (Polanyi, 1958: "we know more than we can tell"). We classify tons of images daily without being able to formulate the rules (i.e., without being able to describe the algorithm—assuming there is one) that we used.

Note that to explain to a child how to distinguish a dog from a cat, we would proceed in a way analogous to how we would train a neural network: we would tell them "this is a cat, that is a dog," showing them many examples until they have learned to make the distinction autonomously.

In cognitive psychology, this is called *implicit learning* (Reber, 1967): the child learns a regularity without being explicitly taught it verbally.

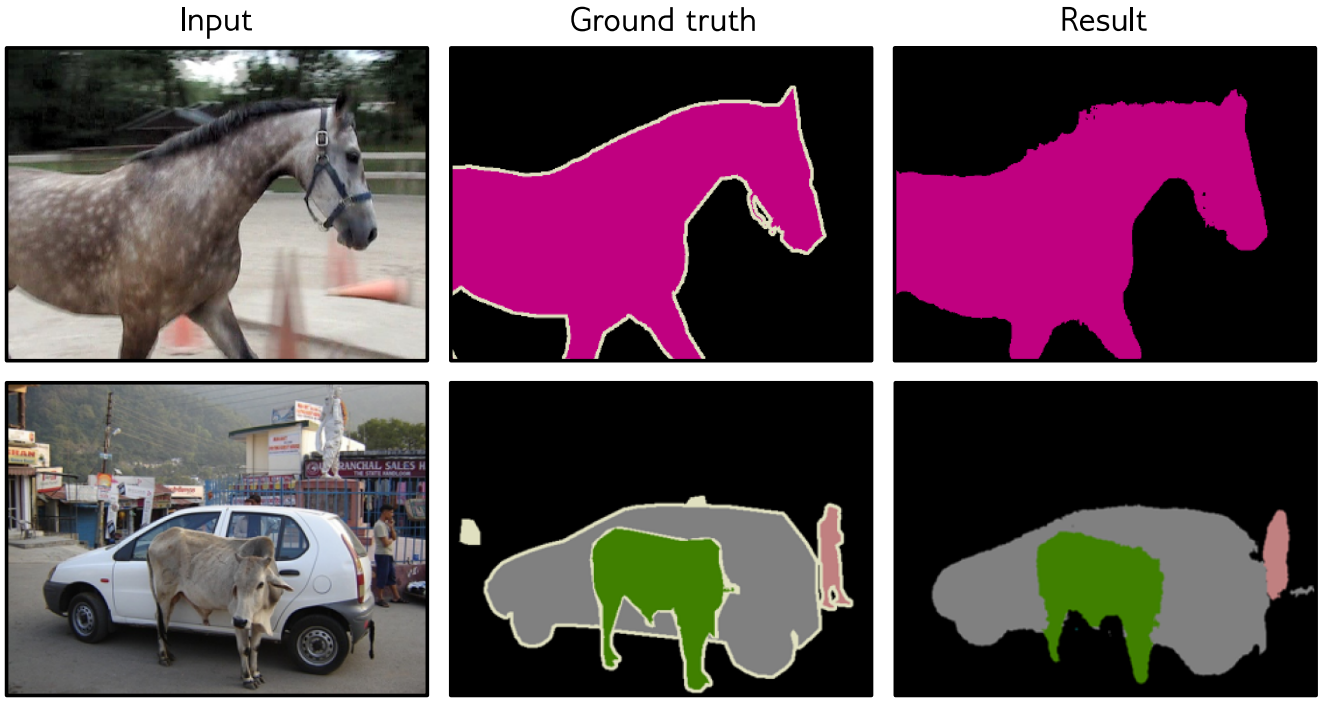
NNs (this should be clear to us at this stage of this course) behave in a similar way: they receive labeled inputs and learn internal regularities that allow them to generalize to new cases. They do not make symbolic rules explicit, but instead contain implicit representations in synaptic weights. Like the child, the network displays a "knowing how" rather than a "knowing that": an operational, not declarative, kind of knowledge. This of course has strong implications when it comes to *interpretability* and *explainability*.

Object detection



(Prince (2023), MIT Press)

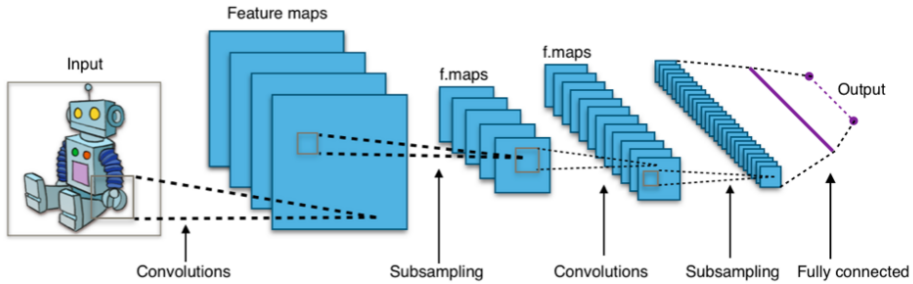
Image segmentation



(Prince (2023), MIT Press)

CNNs

Most of the tasks we mentioned are solved (or, at least, were, before the advent of *Vision Transformers*—a CV extention of the object ChatGPT is based on, which we will study in quite some detail) with a **Convolutional Neural Network** (CNN): the subject matter of this chapter.



CNNs are, arguably, the most important model in the whole of DL. They are completely foundational, as almost *every* modern DL pipieline contains, at some point, something similar to a **convolution**—the central operation a CNN carries out.

Crucially,**CNNs are structured**. They depart from the classical notion of NNs as the unspecialized, self-similar mass of neurons that characterize a fully-connected feed-forward NN (what we call an MLP). They take us to what is standard nowadays in DL: **a NN consisting of blocks, each of which specialized to carry out a specific task**.

2. Why CNNs and not MLPs?

To MLP or not to MLP?

Ockham (or Duns Scotus or maybe Aristotle himself) taught us that we shouldn't make things overly complicated.

This begs the question: *can't we just use an MLP to tackle a CV task such as image classification?*

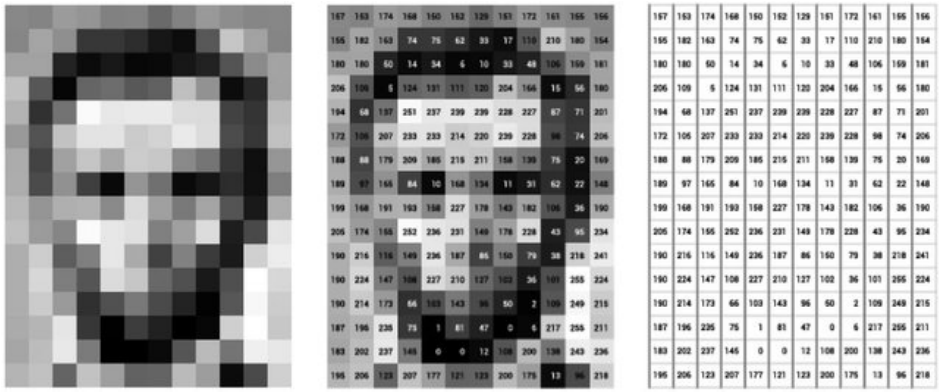
This may work, but it may also not.

Let's see why.

The nature of image data

An image consists of a **matrix of pixels**, where a pixel is just a cell with some intensity values.

In grayscale images, each pixel has a single intensity (gray).



In colored images, it has three: red, green, and blue (RGB). We call these **channels**.

Are MLPs a good idea?

Images can contain a very large number of pixels, making them very high-dimensional data. Treating, as one would do in an MLP, an image as an unstructured collection of pixel values would require an impractically large number of parameters.

Example. Processing a 3-color 100×100 image with an MLP featuring 10 neurons in the first hidden layer would require $(3 \cdot 100^2) \cdot 10 = 300,000$ weights only for the first layer (and we know that we want more than a single hidden layer!).

This is not going to work. We need something smarter than an architecturally-boring MLP.

More importantly, MLPs treat input data as unstructured, i.e., they do not leverage any prior knowledge about the relationships between data points.

Images are different as pixel values are **not** independent: nearby pixels tend to have strongly **correlated values** (indeed: in most cases, the transition from pixel to pixel is smooth).

Such a **locally-structured nature** provides an **inductive bias** that we can leverage.

3. Historical perspective

Convolutional networks

CNNs are, in essence, the brain child of Yann LeCun. The most important incarnation of his work is LeNet-5 (1998, with earlier versions dating back to 1989), which sparked a renewed interest in NN within the whole field of AI.

LeNet is historically important for three reasons: 1. it was one of the first successful applications of backprop; 2. it was the first NN deployed at industrial scale (reading handwritten ZIP codes for the US Postal Service in 1989, and read check amounts for the banking system in 1998), and 3. it established the architectural blueprint that many follow-up works followed.

(Yann LeCun (right)—Photo taken at ICLR2026)

The 1989 paper: LeCun, Boser, Denker, Henderson, Howard, Hubbard, Jackel (1989). *Backpropagation applied to handwritten ZIP code recognition.* Neural Computation, 1(4), 541–551.

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel
AT&T Bell Laboratories Holmdel, NJ 07733 USA

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

The 1998 (LeNet-5) paper: LeCun, Bottou, Bengio, Haffner (1998). *Gradient-based learning applied to document recognition.* Proceedings of the IEEE, 86(11), 2278–2324.

Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner

Abstract—

Multilayer Neural Networks trained with the backpropagation algorithm constitute the best example of a successful Gradient-Based Learning technique. Given an appropriate network architecture, Gradient-Based Learning algorithms can be used to synthesize a complex decision surface that can classify high-dimensional patterns such as handwritten characters, with minimal preprocessing. This paper reviews various methods applied to handwritten character recognition and compares them on a standard handwritten digit recognition task. Convolutional Neural Networks, that are specifically designed to deal with the variability of 2D shapes, are shown to outperform all other techniques.

Real-life document recognition systems are composed of multiple modules including field extraction, segmentation, recognition, and language modeling. A new learning paradigm, called Graph Transformer Networks (GTN), allows such multi-module systems to be trained globally using Gradient-Based methods so as to minimize an overall performance measure.

Two systems for on-line handwriting recognition are described. Experiments demonstrate the advantage of global training, and the flexibility of Graph Transformer Networks.

A Graph Transformer Network for reading bank check is also described. It uses Convolutional Neural Network character recognizers combined with global training techniques to provides record accuracy on business and personal checks. It is deployed commercially and reads several million checks per day.

Keywords— **Neural Networks, OCR, Document Recognition, Machine Learning, Gradient-Based Learning, Convolutional Neural Networks, Graph Transformer Networks, Finite State Transducers.**

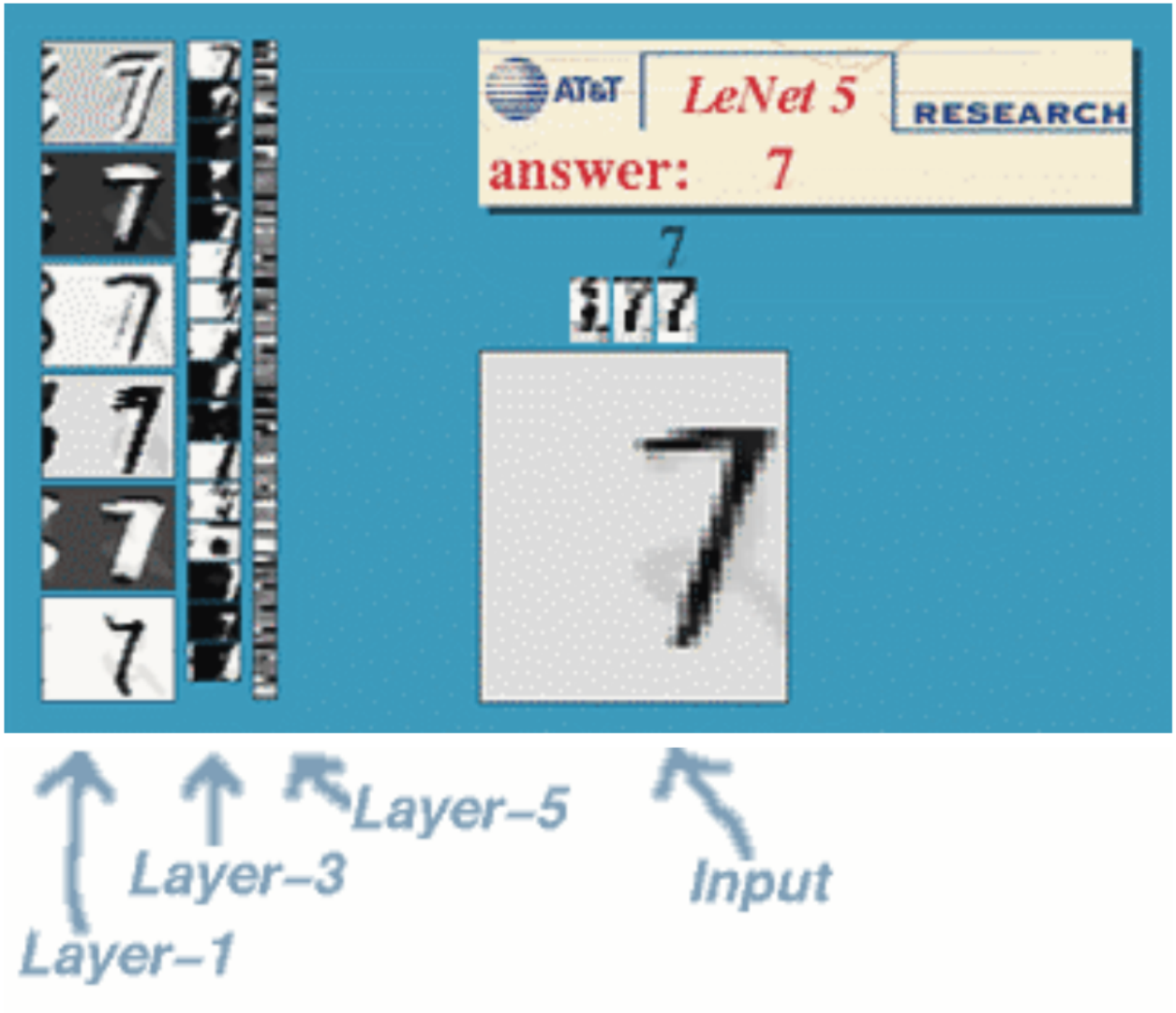
I. Introduction

Over the last several years, machine learning techniques, particularly when applied to neural networks, have played an increasingly important role in the design of pattern recognition systems. In fact, it could be argued that the availability of learning techniques has been a crucial factor in the recent success of pattern recognition applications such as continuous speech recognition and handwriting recognition.

The main message of this paper is that better pattern recognition systems can be built by relying more on automatic learning, and less on hand-designed heuristics. This is made possible by recent progress in machine learning and computer technology. Using character recognition as a case study, we show that hand-crafted feature extraction can be advantageously replaced by carefully designed learning machines that operate directly on pixel images. Using document understanding as a case study, we show that the traditional way of building recognition systems by manually integrating individually designed modules can be replaced by a unified and well-principled design paradigm, called *Graph Transformer Networks*, that allows training all the modules to optimize a global performance criterion.

Since the early days of pattern recognition it has been known that the variability and richness of natural data, be it speech, glyphs, or other types of patterns, make it

A beatiful illustration form LeCun's webpage from the 90s of what a CNN learns across its layers: http://yann.lecun.com/exdb/lenet/index.html

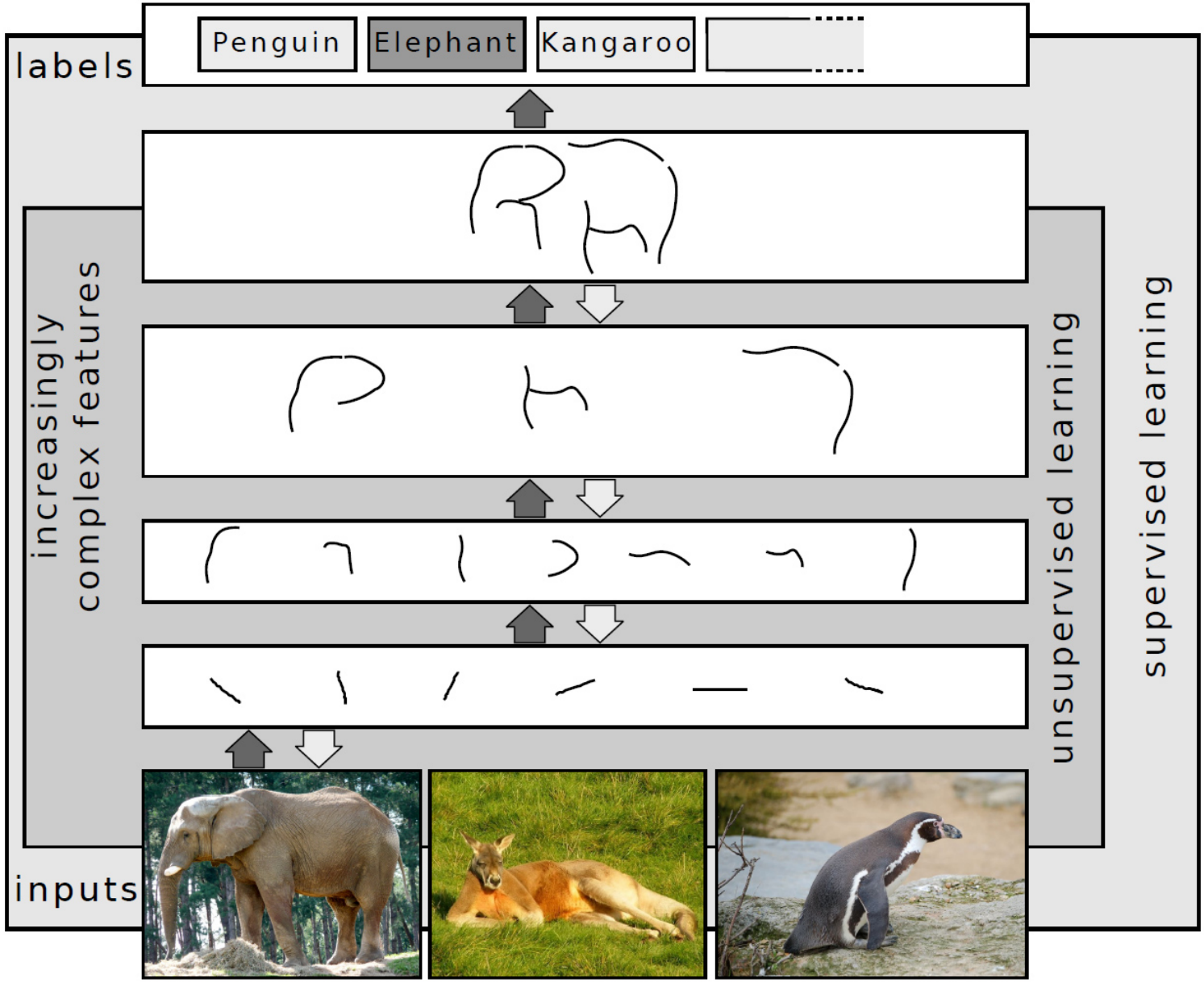


Hubel and Wiesel's work

CNNs originate in studies of the mammalian (a cat's) visual system by David Hubel and Torsten Wiesel (Nobel Prize in Physiology or Medicine in 1981) from the 1960s.

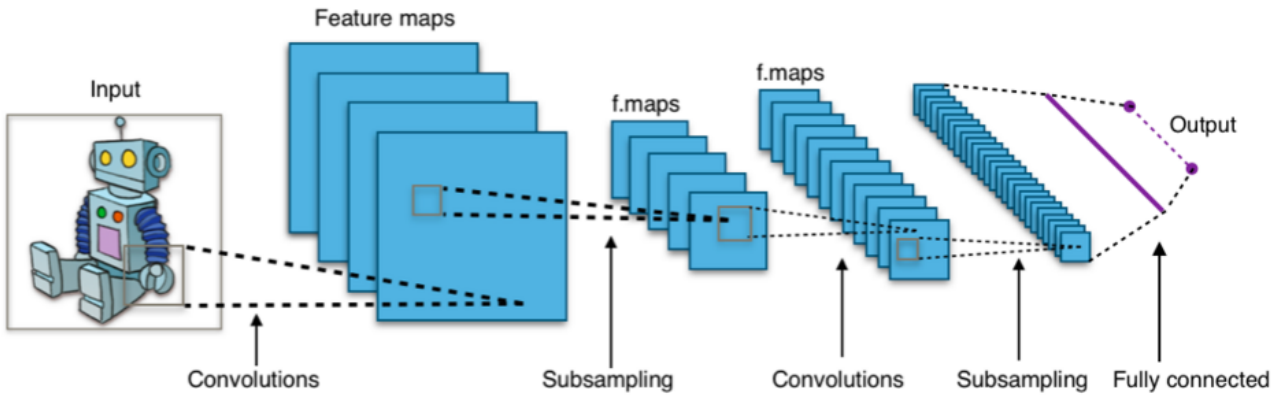
In addition to **photoreceptors** (which activate in response to light hitting a single "pixel" of the visual field—a neuron in the retina), Hubel and Wiesel discovered **a hierarchy of detector neurons**. Some (simple cells) activate in the presence of edges with a precise orientation; others (complex cells) in the presence of more complex shapes and independently of their position.

- *Retinal neurons* (photoreceptive) are sensitive to local variations in luminance
- Those in the *primary visual cortex* (V1) are sensitive to edges and orientations (these are both simple and complex cells).
- Those in *higher areas* (V2, V4) integrate responses from previous levels, activating in the presence of corners, curves, and textures (complex cells).
- Neurons in the *inferotemporal cortex* (IT) respond to complex shapes, relatively invariant to position, scale, and rotation.


labels
Penguin
Elephant
Kangaroo
increasingly
complex features
unsupervised learning
supervised learning
inputs

Hubel and Wiesel's work is crucial because it shows that identifying the *presence* of a **pattern** in the visual field (e.g., a digit) is partly independent of its position and often also of small rotations.

CNNs mirror this idea by learning to recognize **local patterns** (e.g., edges, corners, curves) almost independently of their position in the image and to combine them into increasingly abstract representations across the network's layers to, ultimately, identify the image's content.

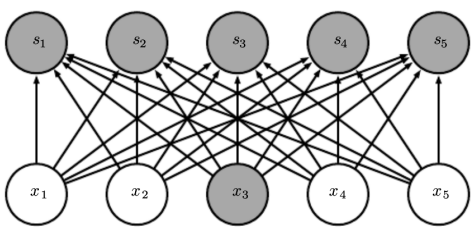


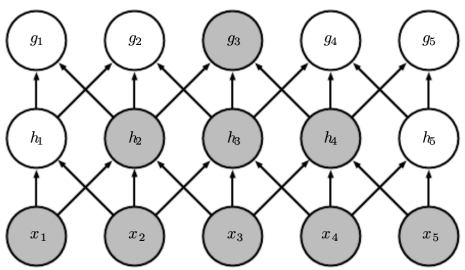
Sparsity and receptive fields in CNNs

In CNNs, the idea of detecting local patterns which are then composed hierarchically translates into the notion of **receptive field**.

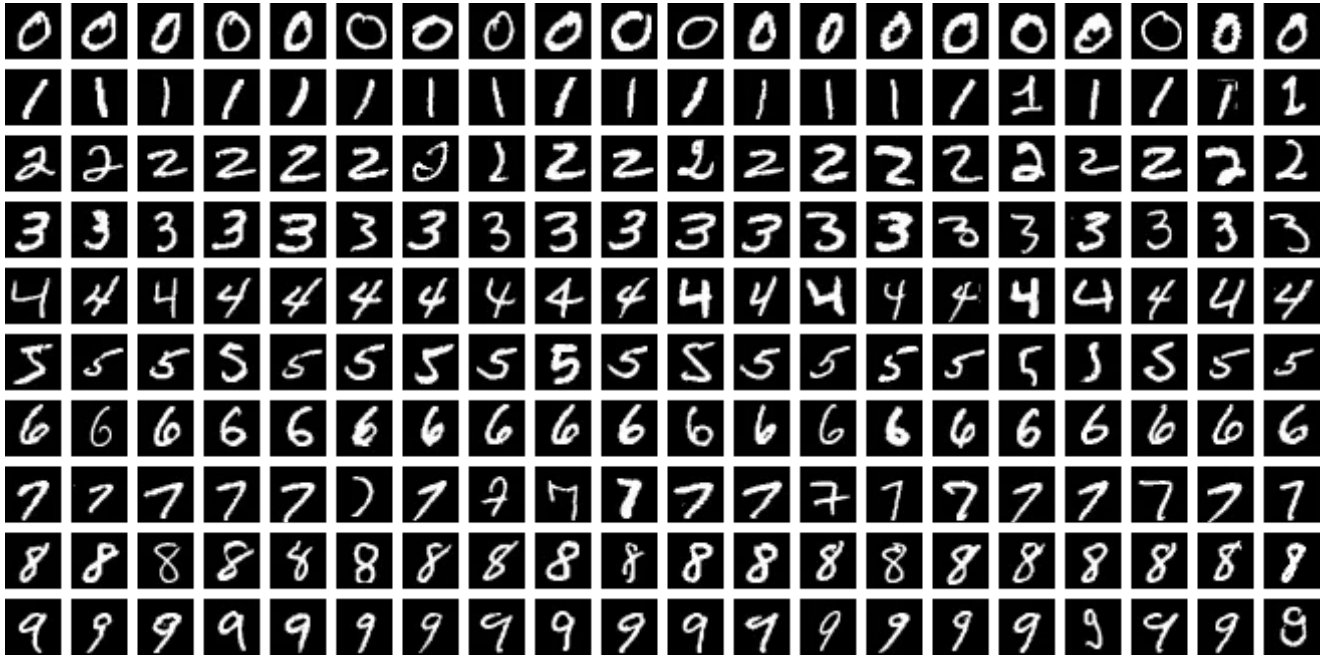
Unlike a fully connected layer, where each output neuron sees the entire input, a convolutional neuron sees only a small local patch of the image: its **receptive field**. This drastically reduces parameters and computation, while at the same time promoting generalization and *invariance* (see further) to where a pattern is contained in an image.

As we stack layers, receptive fields **grow**: a neuron in layer 2 sees a patch of layer-1 outputs, each of which already aggregates a patch of the input. After a few layers, a single neuron is influenced by a wide region of the original image. Simple features (edges) at the bottom; complex structures (parts, objects) at the top, in line with the V1 $\rightarrow$ V2 $\rightarrow$ IT hierarchy.





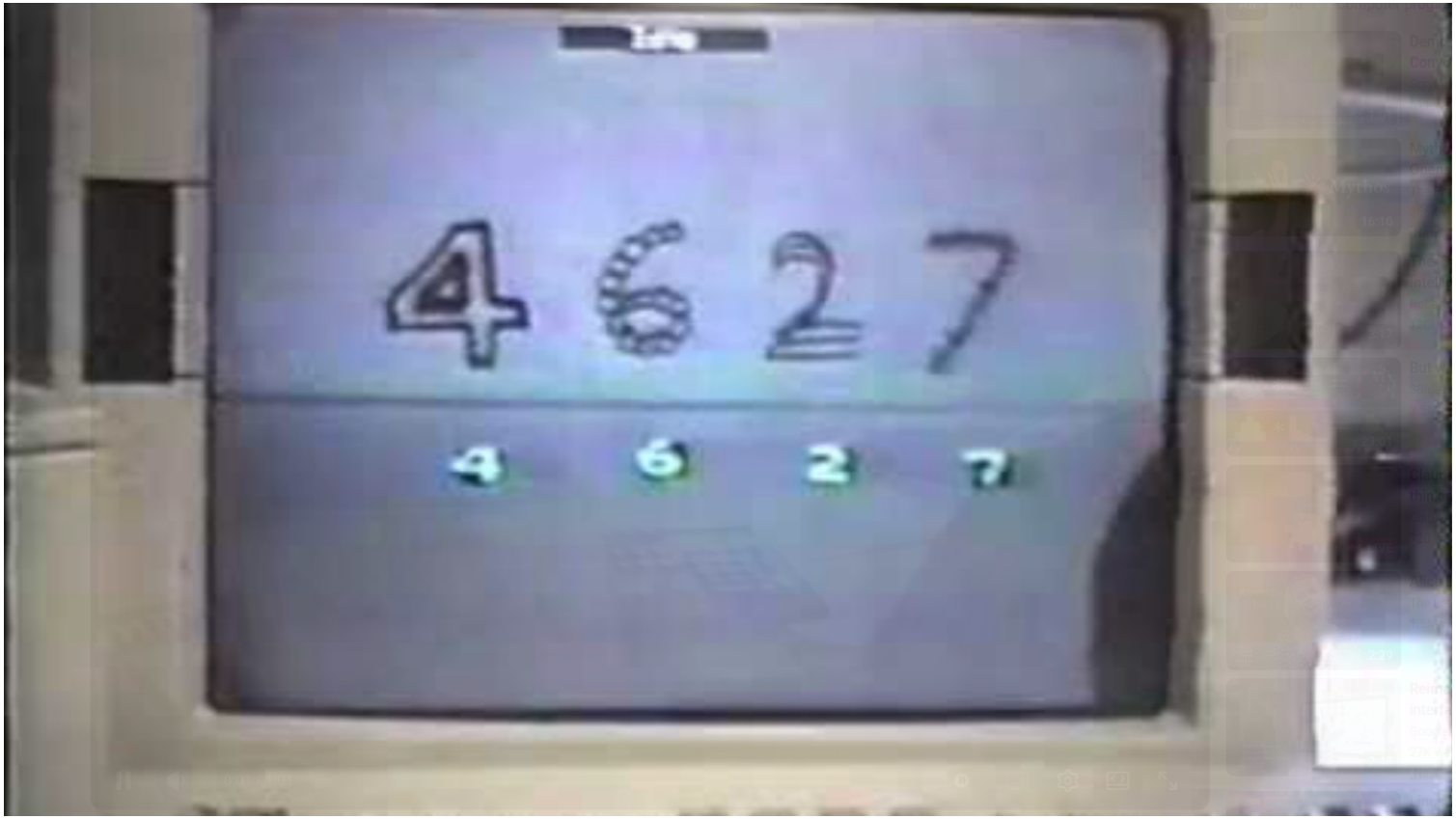
The MNIST dataset and OCR



In the photo, the **MNIST dataset** (curated by LeCun in the 1990s). It contains 28×28-pixel grayscale images of handwritten digits and became a fundamental benchmark for testing CNNs on the OCR (**Optical Character Recognition**) problem.

It was developed to experiment with LeNet within theNIST project of automatic recognition of ZIP-code diigits for the US postal service.

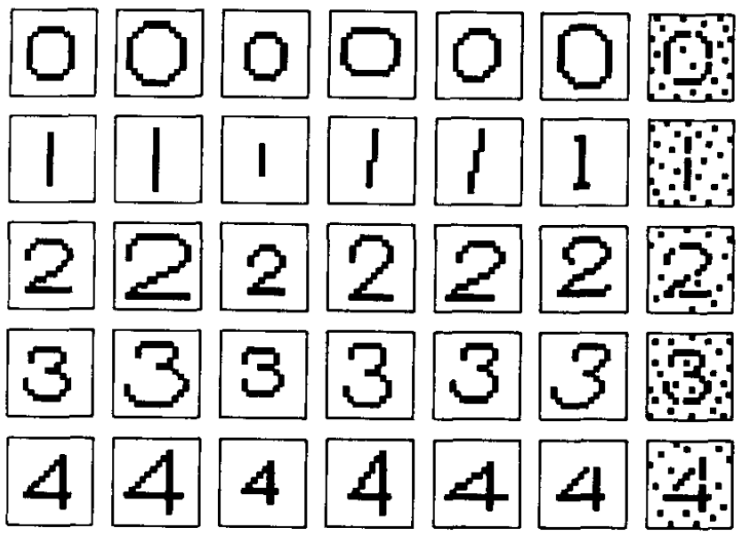
LeCun running LeNet: https://www.youtube.com/watch?v=FwFduRA_L6Q



Neocognitron, 1980

Hubel and Wiesel's work inspired many of the early connectionist vision models (e.g., Marr and Poggio, 1976) in neuroscience.

A predecessor of LeNet is the **neocognitron** by Kunhiko Fukushima (1980), a NN designed specifically to serve as a model of the visual cortex.



Although architecturally very similar to LeCun's network, Fukushima lacked an effective training algorithm (backprop). He, though, introduced the ReLU activation function (as we said).

4. 1D convolution

The convolution operator

CNNs are rooted in the mathematical operator of **convolution**.

Let's start with the 1D case.

Mathematically, 1D **convolution** is defined as an integral (or a summation in the discrete case).

Given an **input** function $x(t)$ and a weighting function $\omega(t)$, also known as **kernel**, their convolution is the function $z(t)$:

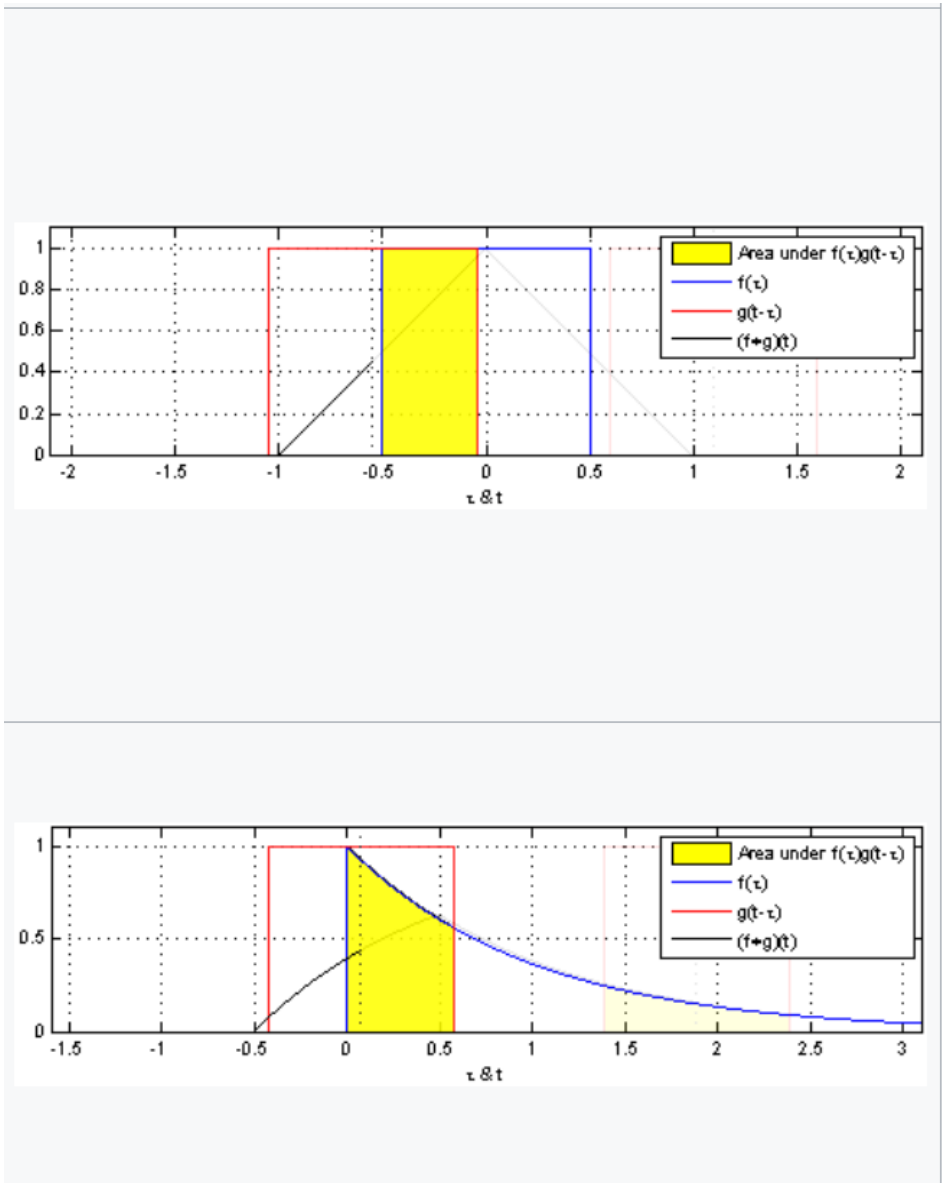
$$z(t) = x(t) * \omega(t) = \int x(\tau)\omega(t-\tau)d\tau$$

In the discrete setting, we have:

$$z(t) = x(t) * \omega(t) = \sum_{\tau=-\infty}^{\infty} x(\tau)\omega(t-\tau)$$

Note: in CNNs, what we compute is *cross-correlation* (we don't actually flip the kernel—it doesn't make a huge difference since, as we will see, the kernel is learned).

A visual (animated) illustration of the convolution integral in action: https://en.wikipedia.org/wiki/Convolution

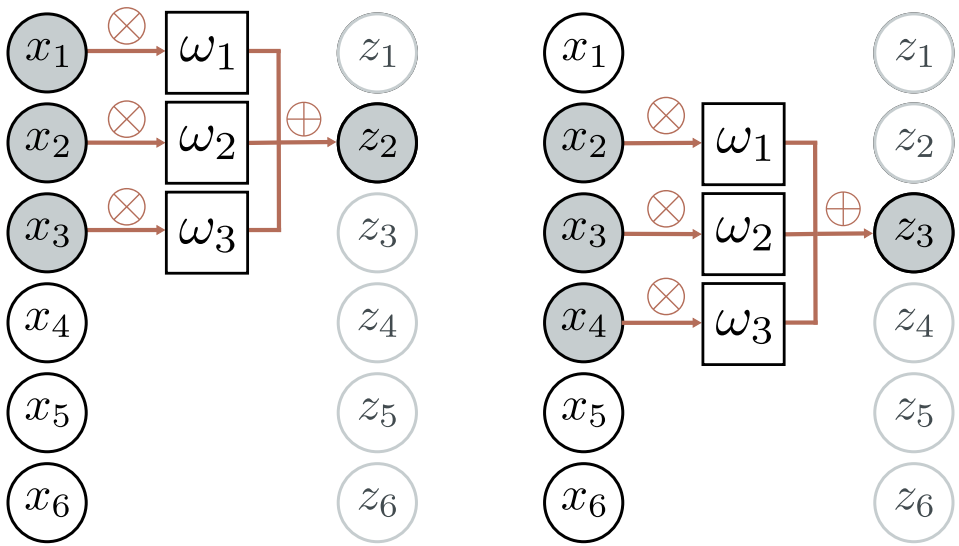


1D CNNs

1D convolutions are heavily used on **time series** or, more generally, univariate **signals**.

Let's see a discrete (as in all NNs) example with a kernel function of "size" 3 (i.e., that takes a nonzero value only for 3 inputs): $\omega = (\omega_1, \omega_2, \omega_3)^\top$.

Let's assume the input vector is $x = (x_1, x_2, \ldots, x_6)^\top$.



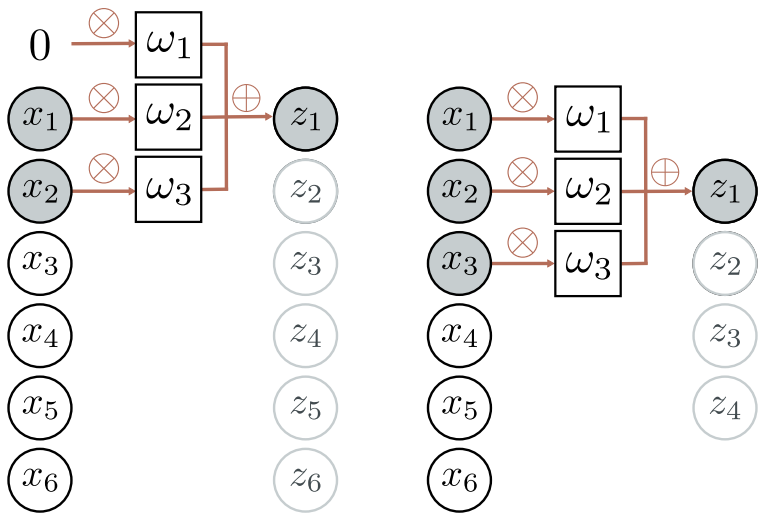
(Prince (2023), MIT Press)

The output is the weighted sum $z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$

Padding strategies

Since convolutions rely on neighboring values, handling the edges of an input sequence requires attention. Two common approaches are:

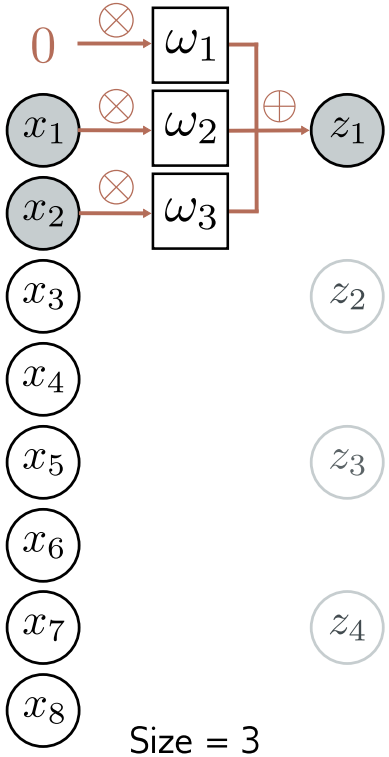
- **Zero-padding**: Extending the input with zeros ensures that the output length remains unchanged.
- **Valid convolution**: The kernel only operates where complete data is available (i.e., we start with an "offset"); this reduces the output size but avoids artificial padding. Warning: this changes the dimension of the output!



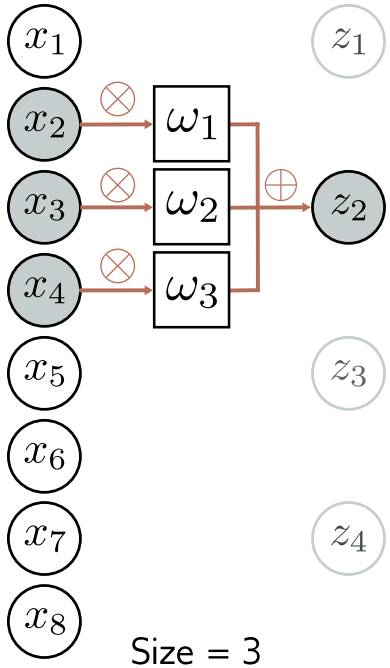
(Prince (2023), MIT Press)

Stride, kernel size, and dilation

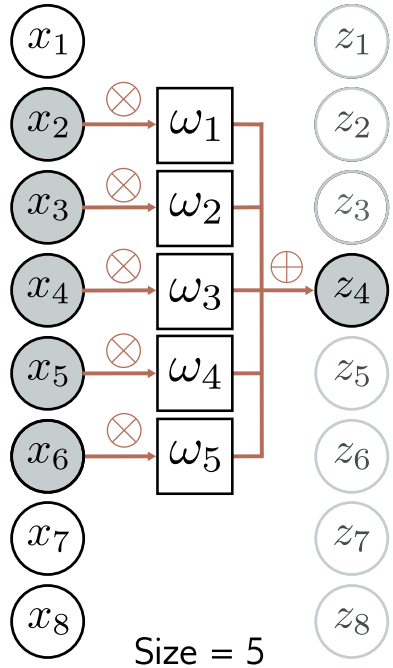
- **Stride**: dictates how much the kernel shifts at each step (the default is 1). A larger stride reduces the output size.
- **Kernel size**: controls the region of influence at each step; larger kernels capture bigger patterns.
- **Dilation**: adds a gap between kernelized elements, expanding the receptive field while keeping parameter count low and somewhat subsampling the input.



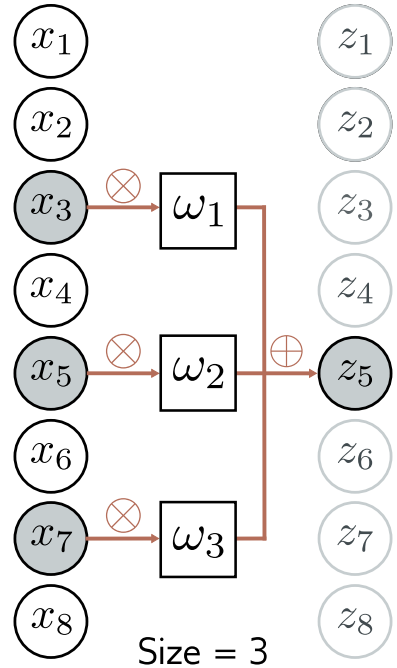
Size = 3
Stride = 2
Dilation = 0



Size = 3
Stride = 2
Dilation = 0



Size = 5
Stride = 1
Dilation = 0



Size = 3
Stride = 1
Dilation = 1

(Prince (2023), MIT Press)

1D convolution: numerical examples (I/IV)

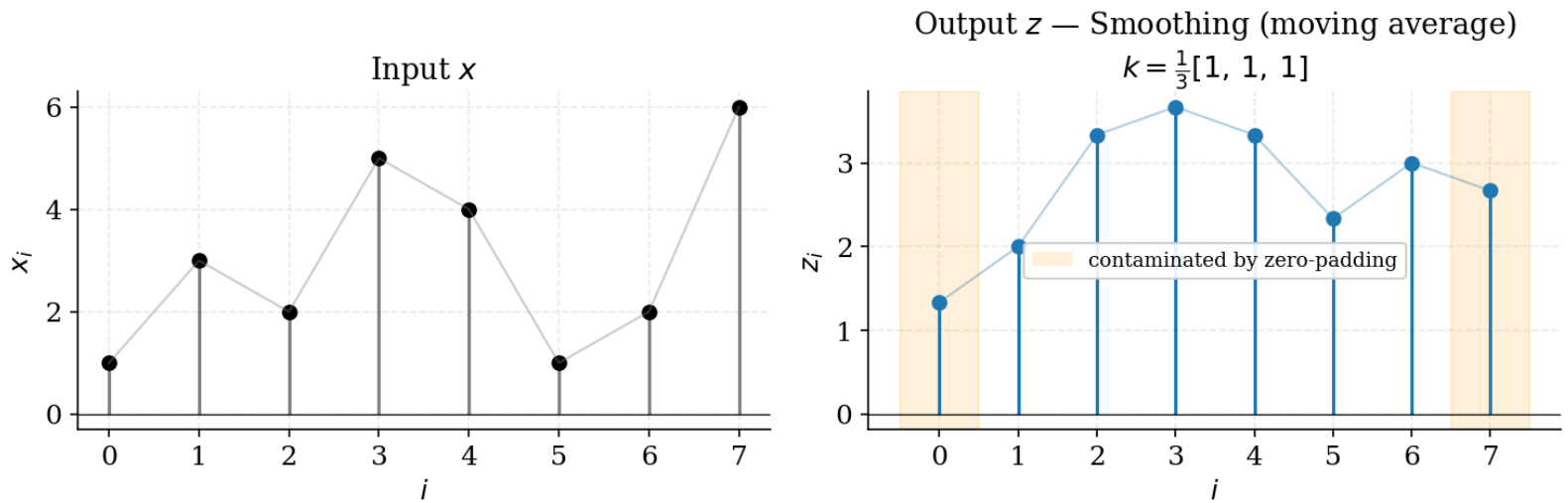
Consider the input signal $x = (1, 3, 2, 5, 4, 1, 2, 6)^\top$. We apply kernels of size 3, with stride 1 and zero-padding to preserve length.

Smoothing kernel (moving average): $k = \frac{1}{3}(1, 1, 1)^\top$:

$$z_i = \frac{1}{3}(x_{i-1} + x_i + x_{i+1})$$

$$z \approx (1.33,\ 2.00,\ 3.33,\ 3.67,\ 3.33,\ 2.33,\ 3.00,\ 2.67)^\top$$

Output is a smoother version of the input: high-frequency variations are attenuated (we applied a low-pass filter).



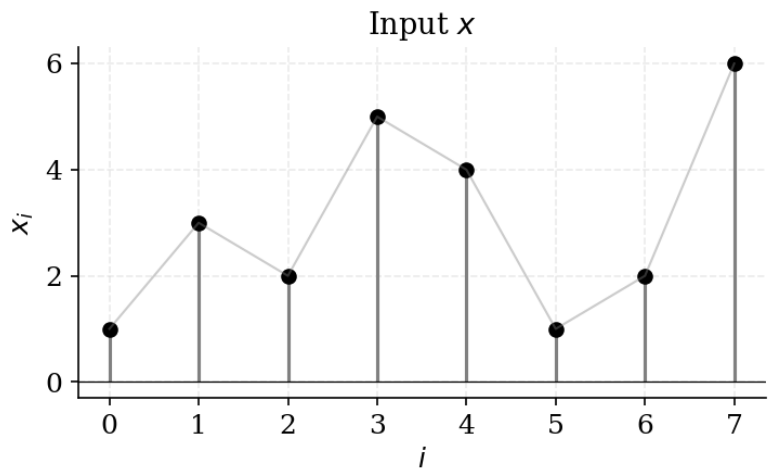
1D convolution: numerical examples (II/IV)

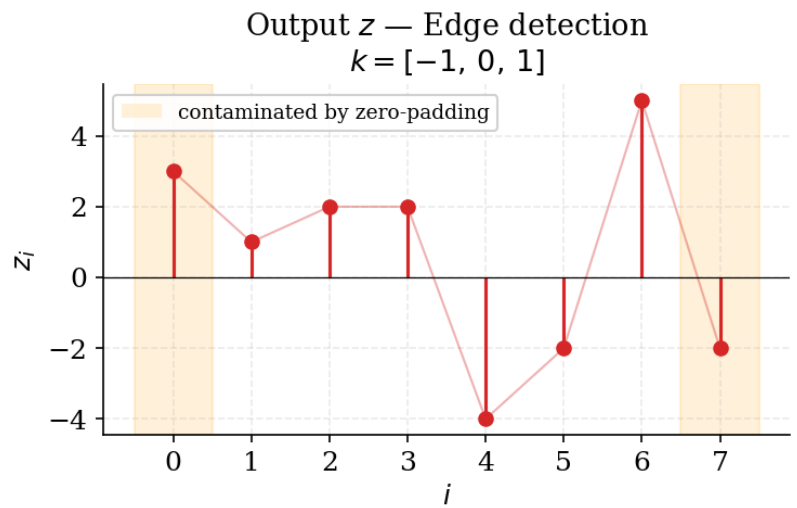
Edge-detection kernel (discrete derivative): $k = (-1, 0, 1)^\top$:

$$z_i = x_{i+1} - x_{i-1}$$

$$z = (3,\ 1,\ 2,\ 2,\ -4,\ -2,\ 5,\ -2)^\top$$

Large $|z_i|$ marks abrupt changes; the sign indicates direction (rising vs. falling).





1D convolution: numerical examples (III/IV)

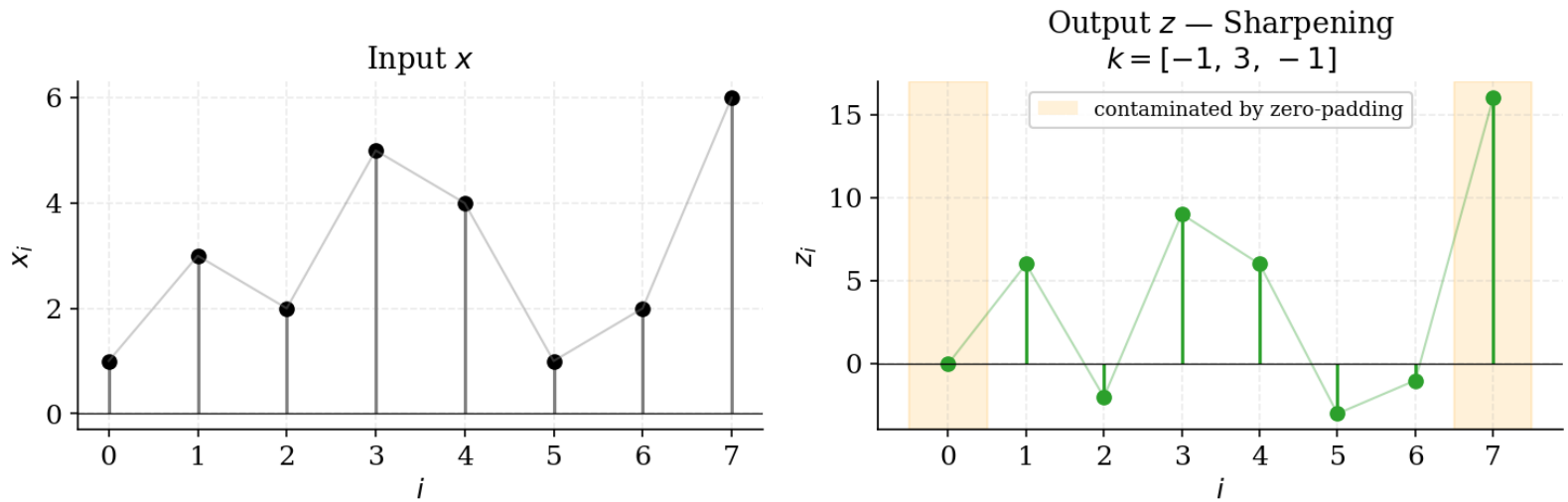
Same input $x = (1, 3, 2, 5, 4, 1, 2, 6)^\top$.

Sharpening kernel: $k = (-1, 3, -1)^\top$:

$$z_i = 3x_i - x_{i-1} - x_{i+1}$$

$$z = (0,\ 6,\ -2,\ 9,\ 6,\ -3,\ -1,\ 16)^\top$$

Emphasizes the central value relative to its neighbors: peaks become more pronounced.



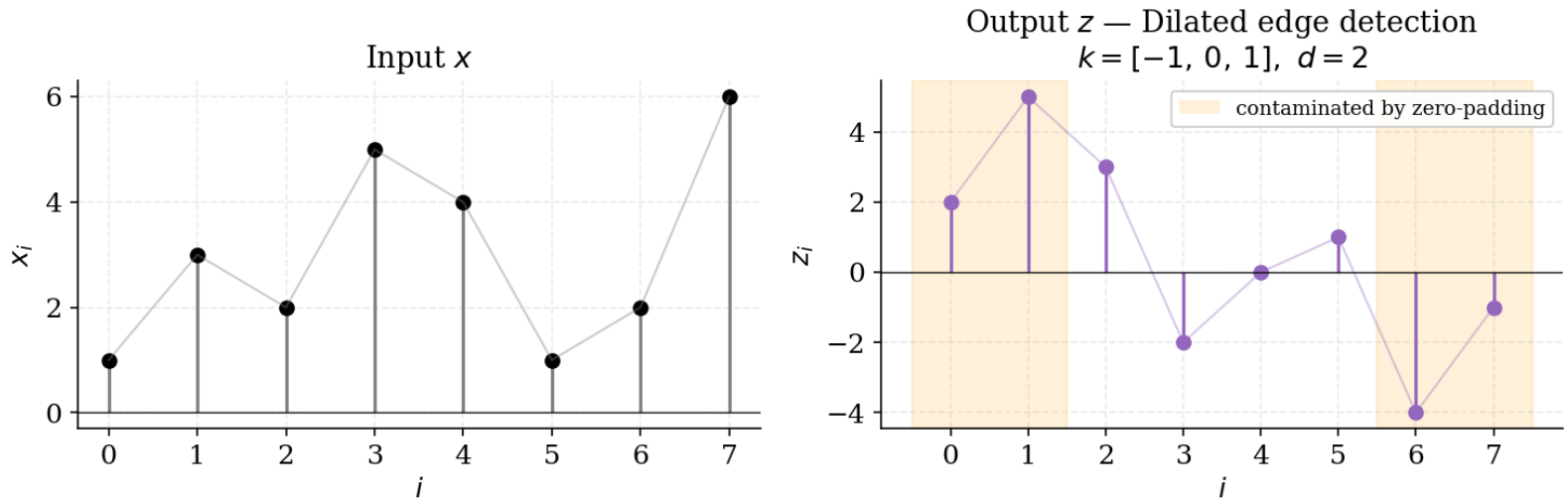
1D convolution: numerical examples (IV/IV)

Dilated edge-detection kernel: $k = (-1, 0, 1)^\top$ with dilation $d = 2$:

$$z_i = x_{i+2} - x_{i-2}$$

$$z = (2,\ 5,\ 3,\ -2,\ 0,\ 1,\ -4,\ -1)^\top$$

Same parameter count as the size-3 edge detector, but the receptive field is now 5: it captures longer-range variations (note the wider zero-padding boundary band).



Takeaway: The kernel *is* the feature detector. In a CNN, these weights are not hand-designed but learned from data.

Convolutional layers and activation functions

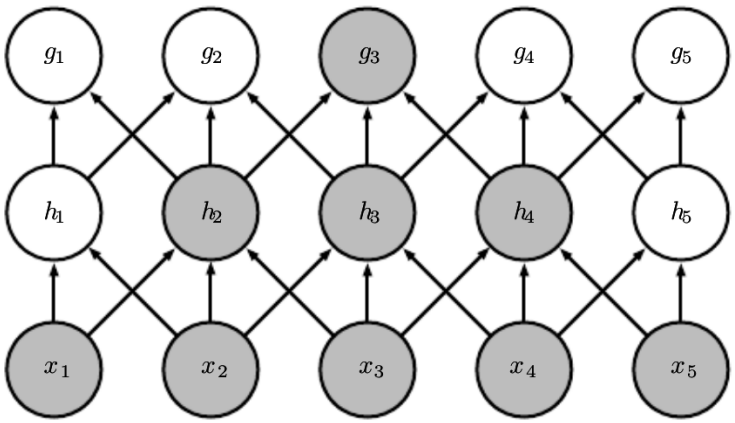
A convolutional **layer** typically includes:

- A **bias** term b, which shifts the activation threshold.
- A **non-linear activation function** Φ, such as ReLU (or tanh in LeNet-5).

Combining these elements with the convolution operation and assuming a kernel of size 3, the (hidden layer) activation h_i is:

$$h_i = \Phi\left(b + \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}\right) = \Phi\left(b + \sum_{j=1}^{3} \omega_j x_{i+j-2}\right).$$

Notice that $\omega_1, \omega_2, \omega_3$ and b are **shared across all positions** i: the *entire layer* uses only 3 weights and 1 bias.

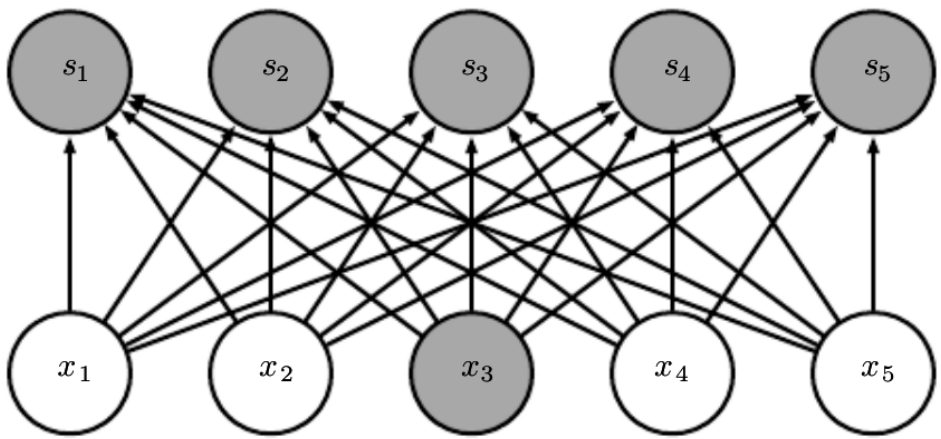


Compare this to a **fully-connected layer** where each h_i takes as input the whole of x; assuming it to be n_{in}-dimensional, we'd have:

$$h_i = \Phi\left(b_i + \sum_{j=1}^{n_{in}} \omega_{ij} x_j\right)$$

i.e., $n_{in}\, n$ weights assuming a hidden layer of size n.

With the previous CNN with a kernel of size 3, the number of weigths is **independent of the input size**.



From hand-crafted to learned filters

The four kernels we saw are only a sample of the "vocabulary" of operations one could carry out. Their role is as follows:

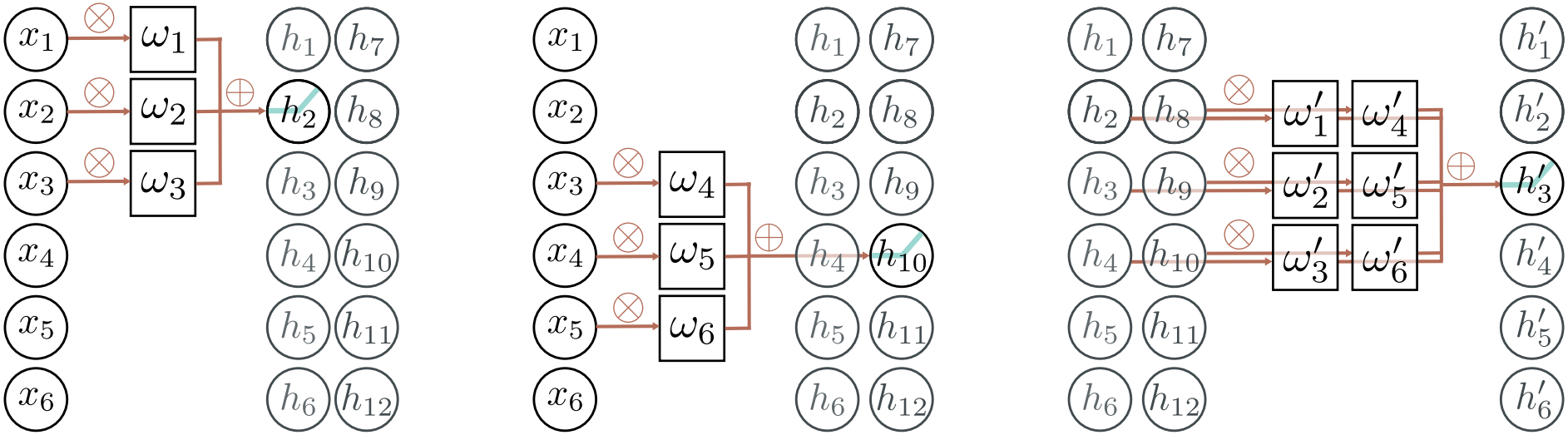
- **Smoothing** suppressed high-frequency variation.
- **Differentiating** detects change.
- **Sharpening** enhances contrast.
- **Dilating** extends reach without extra parameters.

In a CNN, each layer applies its filters to the *output* of the previous one, not to the raw input. The result is a hierarchy of increasing abstractions, such as pixel intensities $\rightarrow$ edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects (this would mirror the V1 $\rightarrow$ V2 $\rightarrow$ IT pathway that Hubel and Wiesel described).

The key to a CNN is that CNN kernels are *learned* to maximize the performance for the downstream task.

Multiple channels

To capture different aspects of the input, one could adopt multiple convolutional filters, in **parallel**. Each filter extracts a unique feature map, which is then stacked into a set of **channels**. For multi-signals (vector functions), we may have natural channels already in the input data.

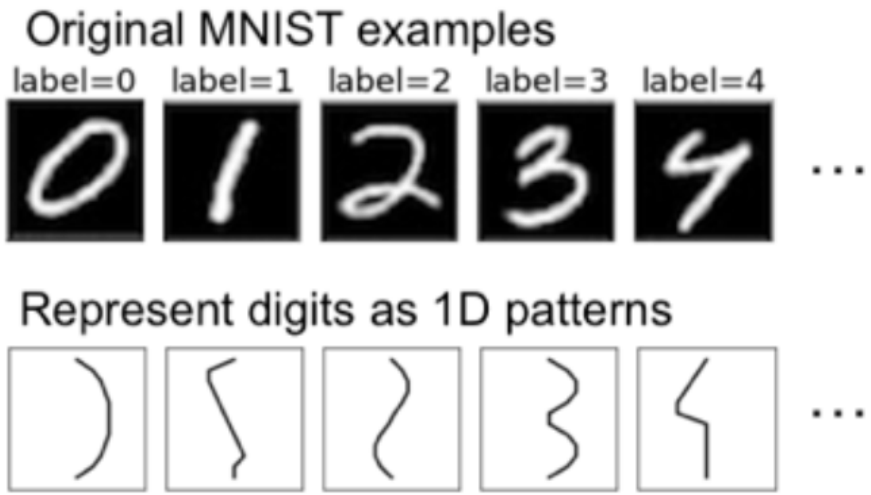


(Prince (2023), MIT Press)

Increasing the dimensionality of the input data (in signal, adding more values per timestamp or, in a picture, adding more channels per pixel) is customary in modern DL. This enables the NN to learn multiple feature representations, improving its ability to model complex patterns in the data.

Example: MNIST-1D

Let's apply a 1D CNN to the MNIST-1D dataset, where each input is a 40-dimensional vector. MNIST-1D features "functionalized" (i.e., turned into a function) versions of hand-written digits.

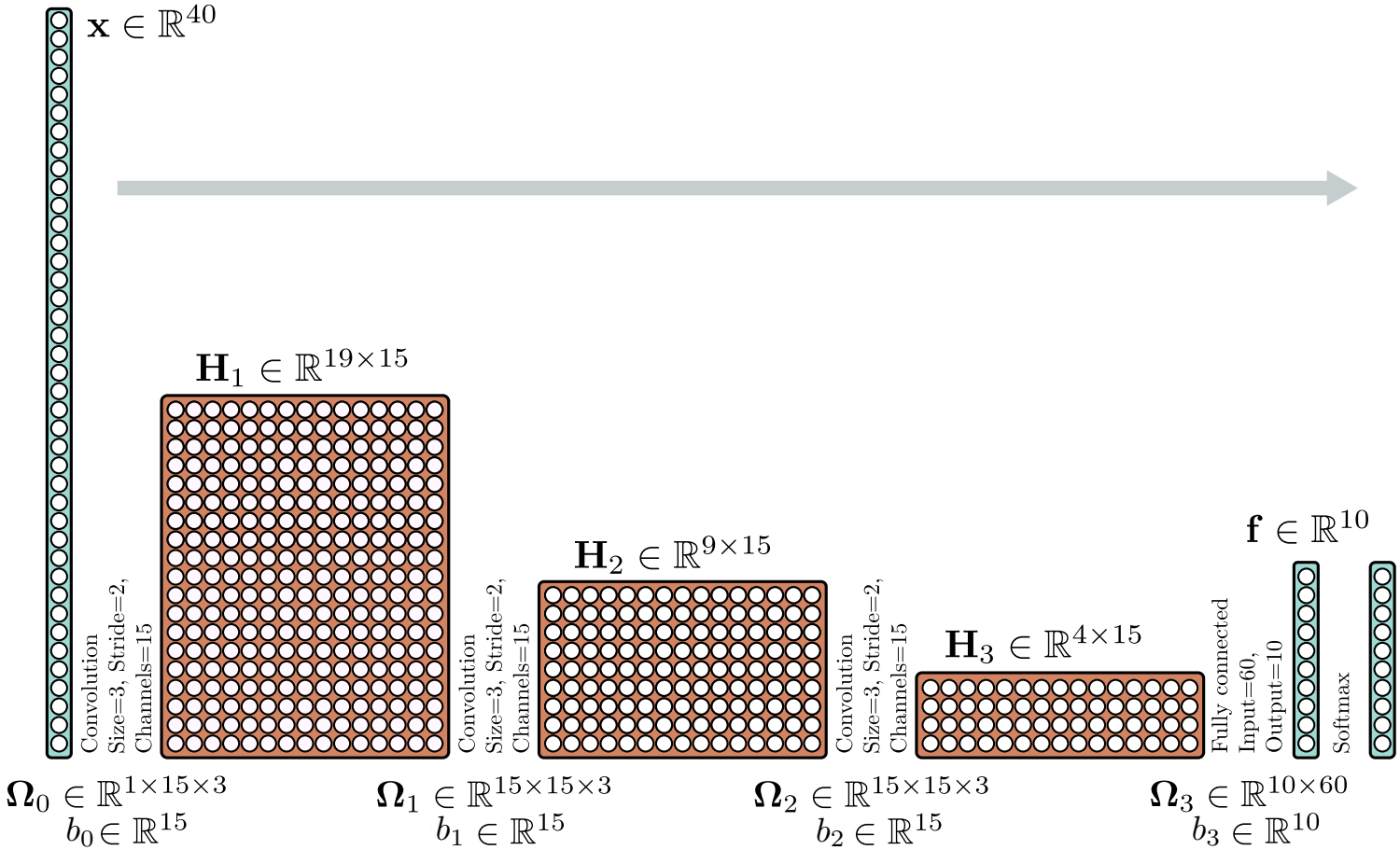


The dataset can be found here: https://github.com/greydanus/mnist1d.

We consider this architecture (results, to follow, taken from Prince (2023)):

- 3 convolutional layers with decreasing dimension and increasing channels
- A fully connected layer that maps the last extracted features to class probabilities (via a softmax—we'll study it in another chapter).

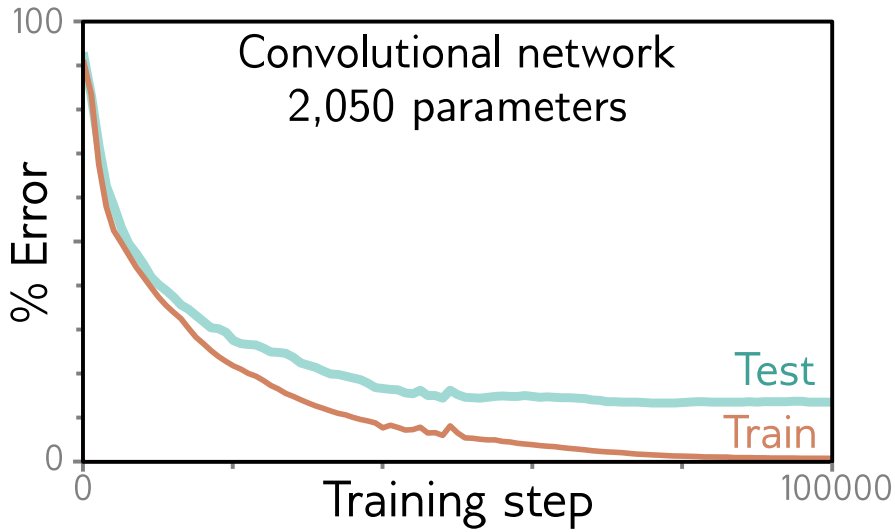
Across the layers, the NN learns more and more features of the image, ultimately leading to a class probability.

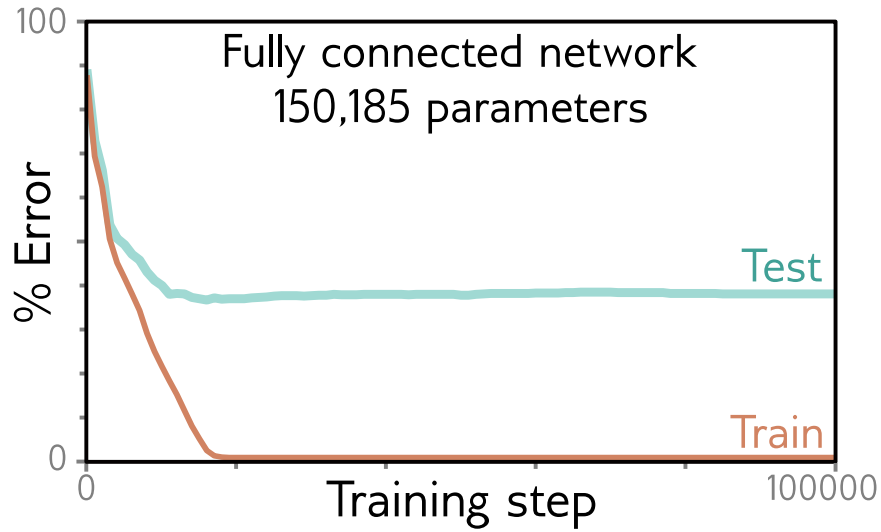


(Prince (2023), MIT Press)

The previous CNN fits the training data with 0 training error and achieves a test error of 17%.

An MLP of the same achieves the same 0 training error faster but generalizes poorly (40% test error).





(Prince (2023), MIT Press)

5. 2D convolution

Convolutional Networks for 2D Inputs

In the case of 2D inputs, the convolutional kernel is also 2D, typically a **square matrix**.

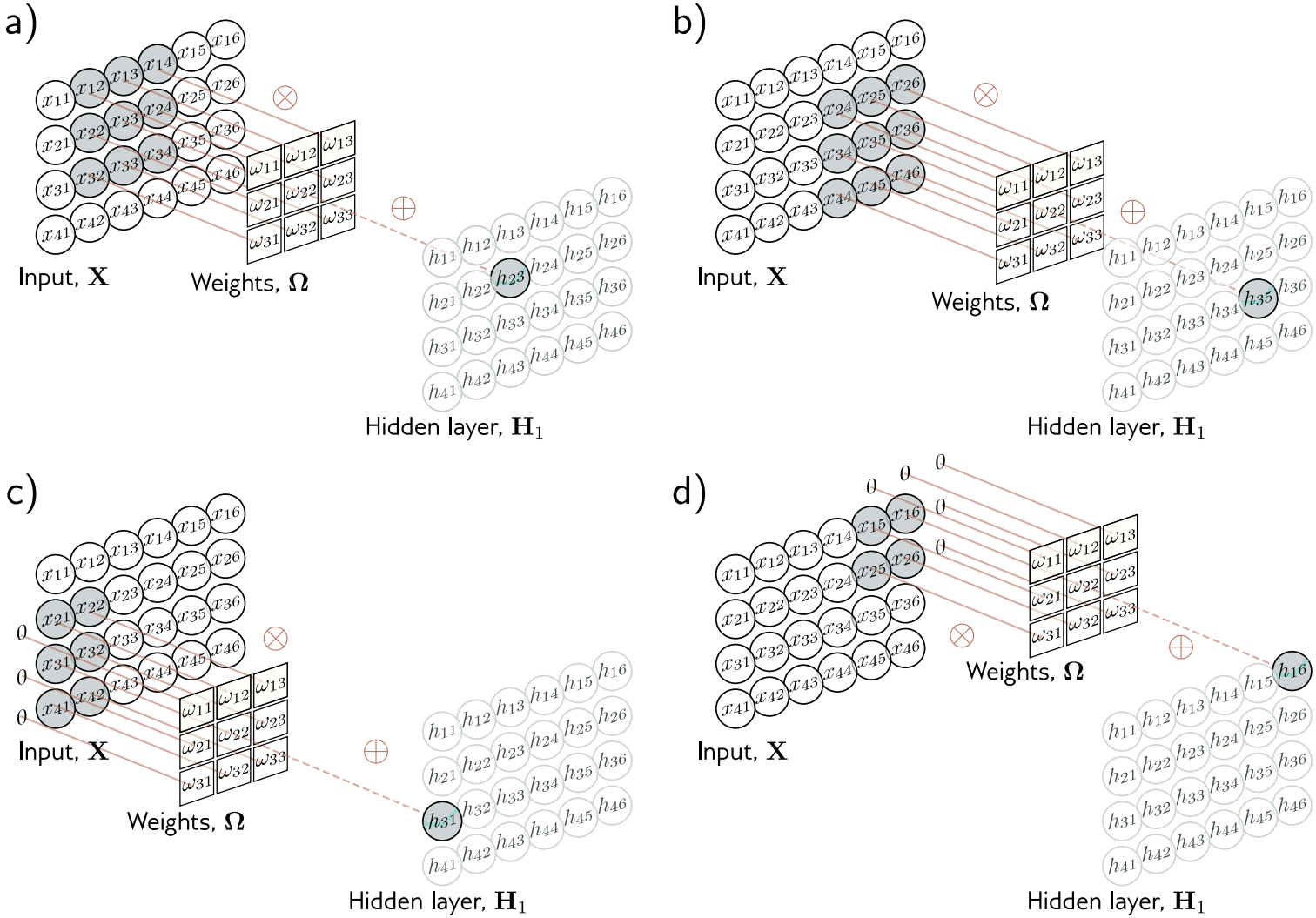
Using a 3×3 kernel $\Omega \in \mathbb{R}^{3\times 3}$ and given an input matrix with elements x_{ij}, the hidden unit activations h_{ij} at each position are:

$$h_{ij} = \Phi\left(b + \sum_{m=1}^{3}\sum_{n=1}^{3} \omega_{mn} x_{i+m-2,j+n-2}\right),$$

where ω_{mn} are the weights of the convolutional kernel. This operation involves a weighted sum over a local 3×3 region of the input.

Let's see an example.

Illustration



(Prince (2023), MIT Press)

Notice how, for 2D applications, we draw the units (neurons) as a matrix, rather than as a vector.

2D convolution: examples

Let's see a few examples of 2D convolution applied to grayscale images, with pixel values in $[0, 255]$ (0 = black, 255 = white).

A word of caution: since convolution sums weighted neighbors at each pixel, the output z_{ij} can be *any* real number—positive, negative, or larger than 255. So the output is no longer "an image" in the same sense as the input—it is a *signed scalar field*.

To display the output "as an image", we adopt the following convention:

$$z_{ij} \longmapsto \min(|z_{ij}|, 255),$$

that is: we take absolute value, then clip to $[0, 255]$.

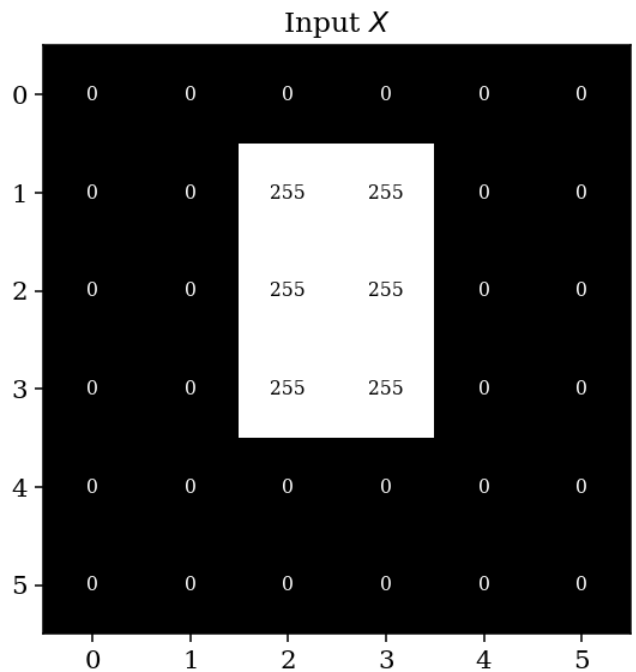
Here's a great interactive example of what we will see: `https://setosa.io/ev/image-kernels/`

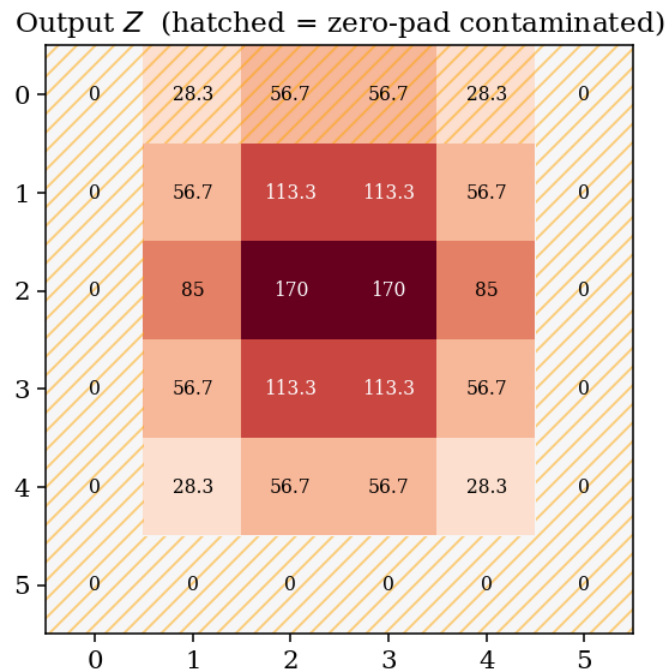
Smoothing

$$k = \frac{1}{9}\,\mathbf{1}_{3\times 3}, \qquad z_{ij} = \frac{1}{9} \sum_{u,v\in\{-1,0,1\}} x_{i+u,\,j+v}$$

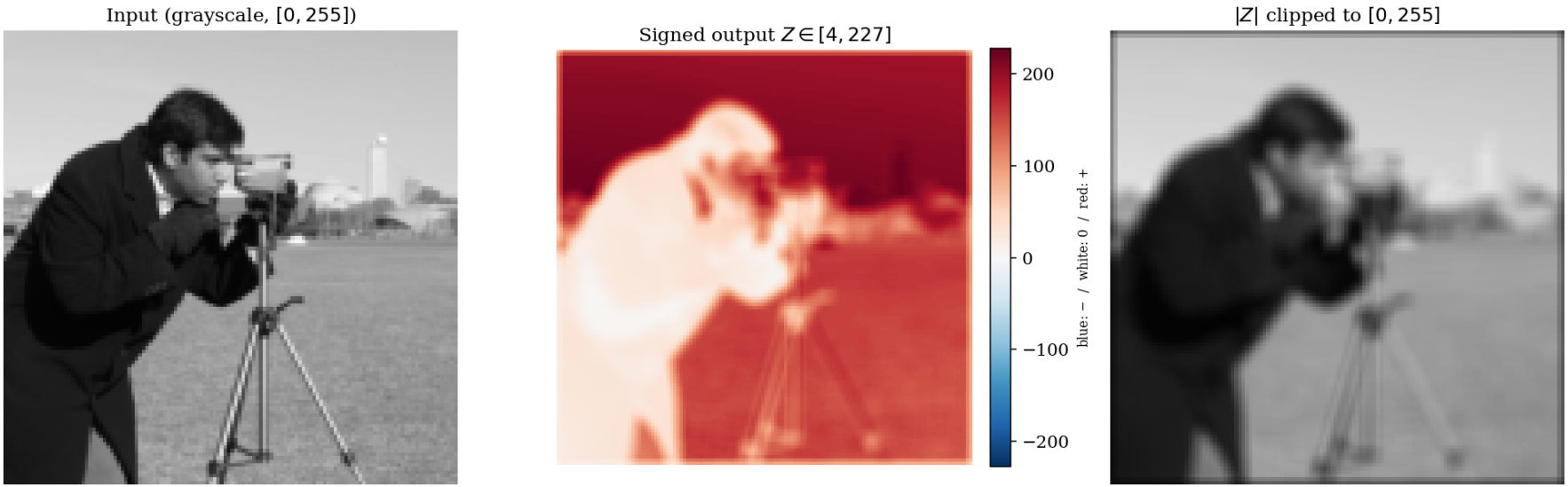
Since the weights in k are non-negative and sum to 1, the output is a *weighted average* of the neighborhood: a *smoothed* version of it.

(Nite: z_{ij} stays bounded by the input range: no signed values, no clipping).





Smoothing with a 5×5 kernel.

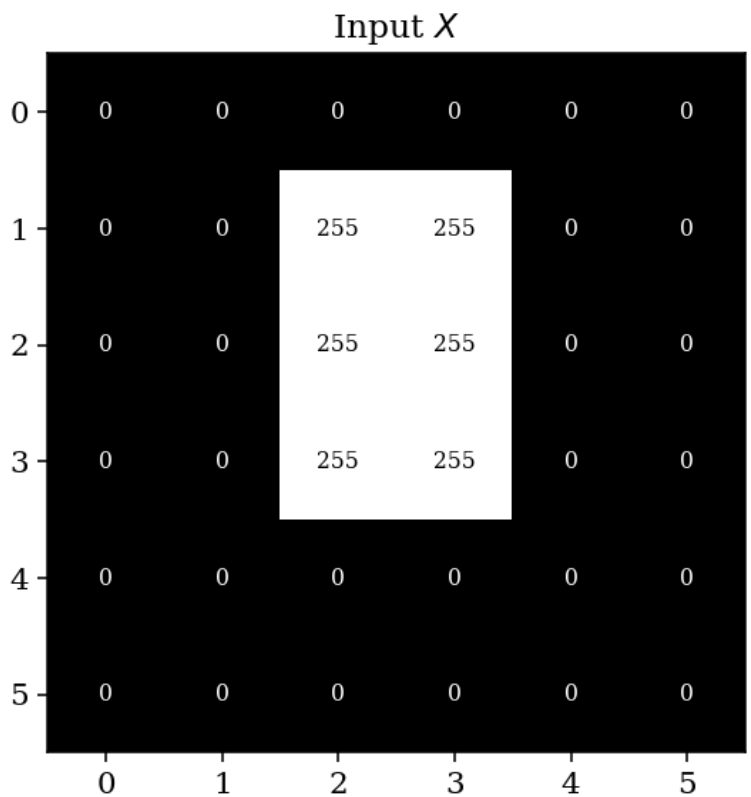


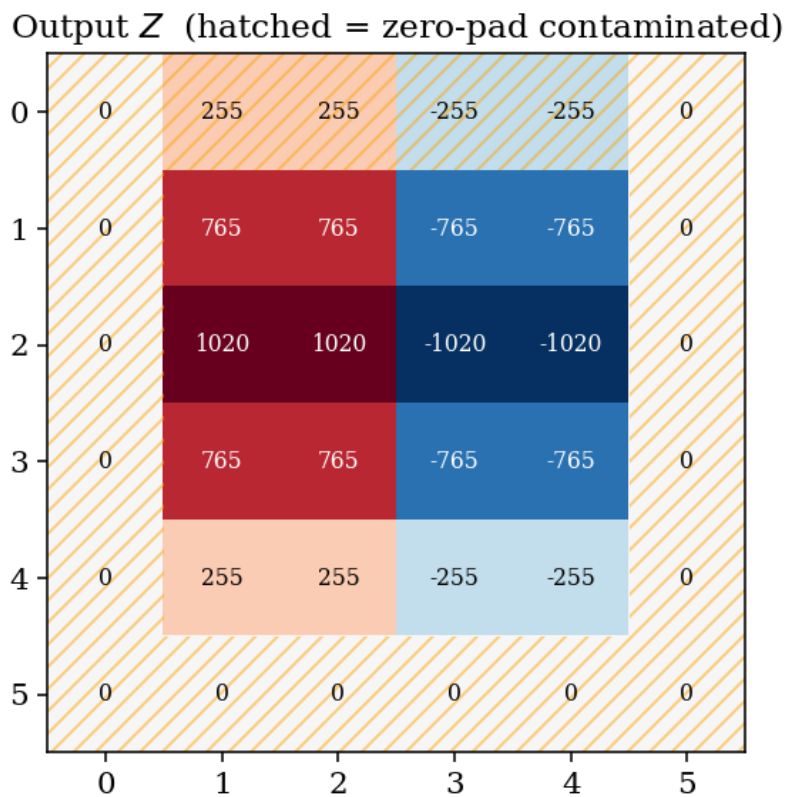
Grass texture, building edges, and facial detail are all attenuated; the cameraman becomes a soft silhouette.

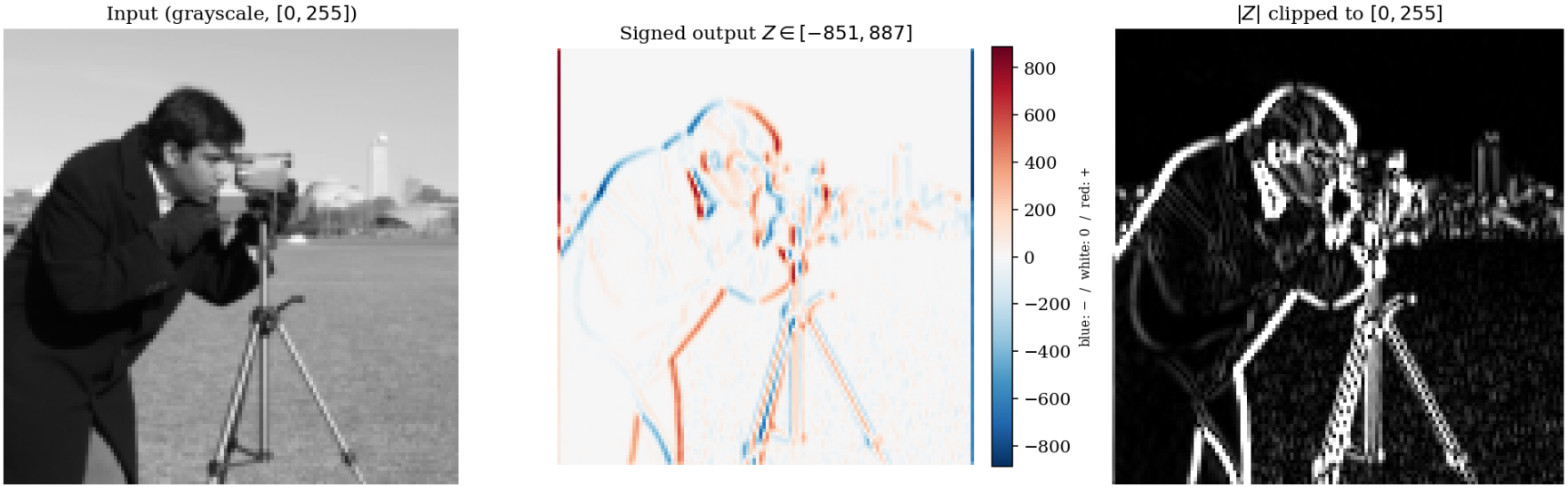
Sobel-x (vertical edges)

$$k = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \qquad z_{ij} = \sum_{u,v \in \{-1,0,1\}} k_{u,v}\, x_{i+u,j+v}$$

Since also here the kernel weights sum to zero, we have $z = 0$ in "flat" regions. The sign indicates direction (red = bright on right, blue = bright on left).







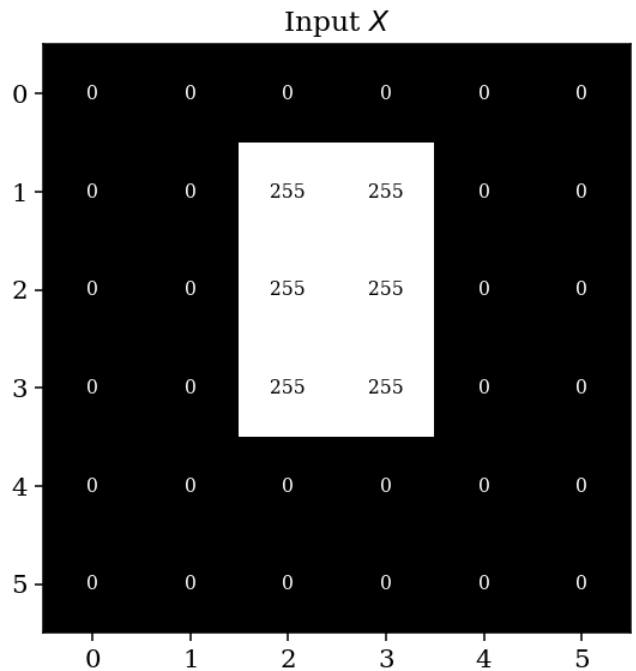
Notice the strong values on tripod legs and cameraman (all vertical features).

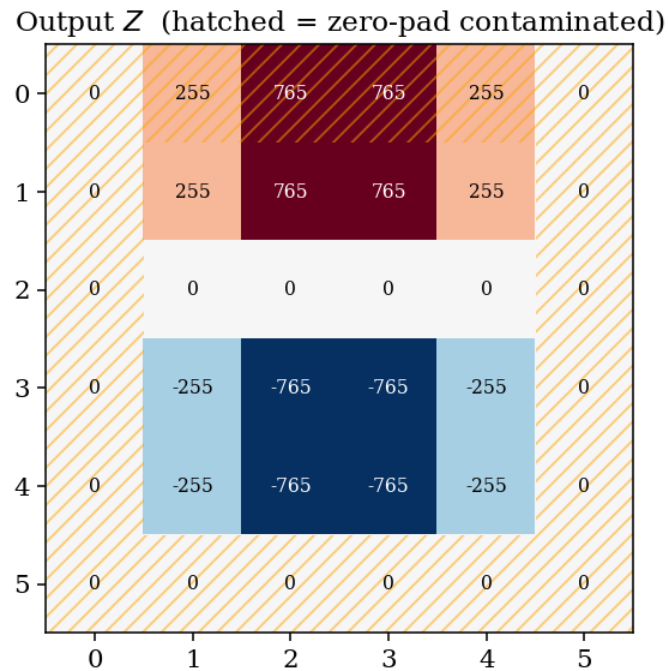
The picture in the middle shows both edge directions (red = dark to light and blue = light to dark).

Sobel-y (horizontal edges)

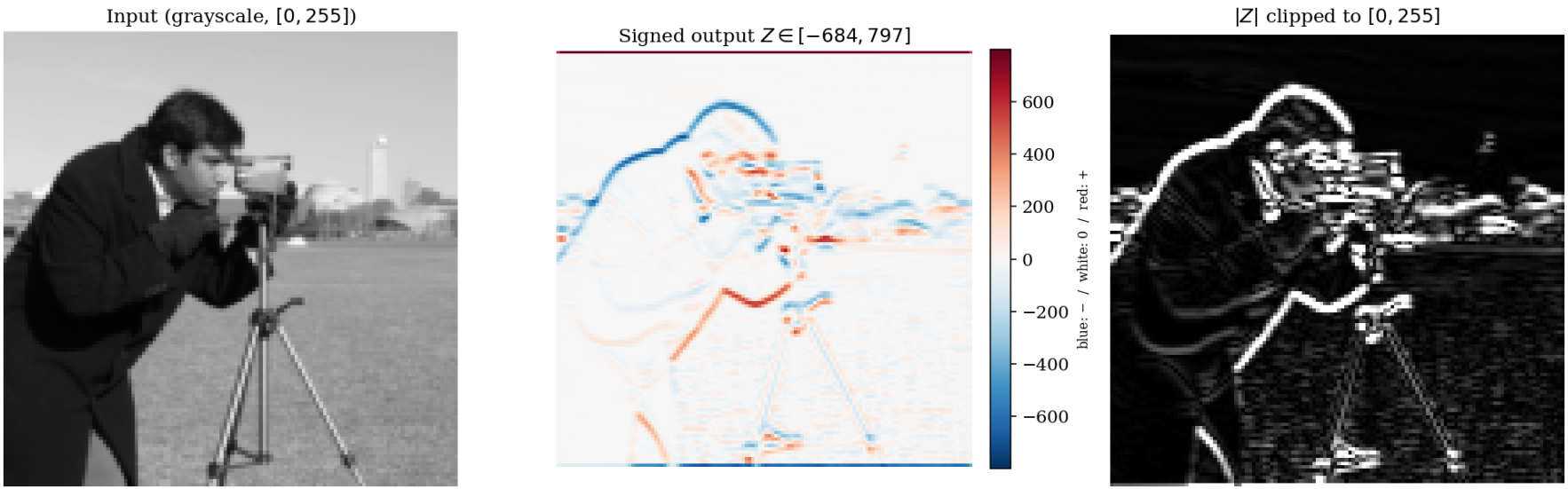
$$k = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}, \qquad z_{ij} = \sum_{u,v \in \{-1,0,1\}} k_{u,v}\, x_{i+u,j+v}$$

Transpose of Sobel-x.





Notice the 0 values in the middle: there is no gradient between the pixel above and below the central ones.



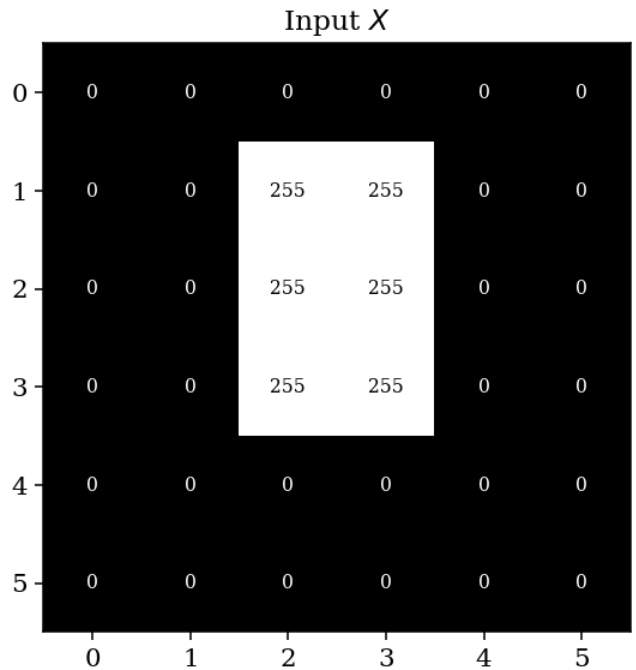
Complementarly to Sobel-x, Sobel-y catches the head's curved top, the horizon, the bottom of the coat (features that Sobel-x misses because they run horizontally).

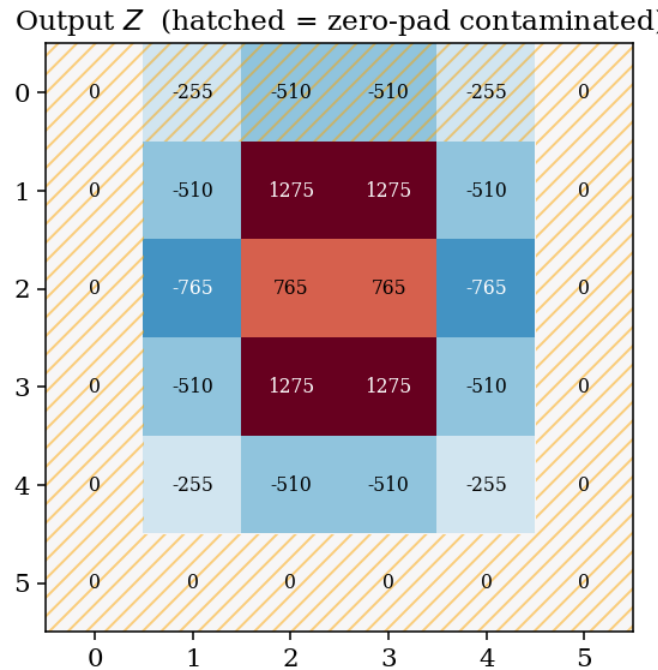
Together, Sobel-x and Sobel-y form a basis for gradient (edge) detection at any orientation.

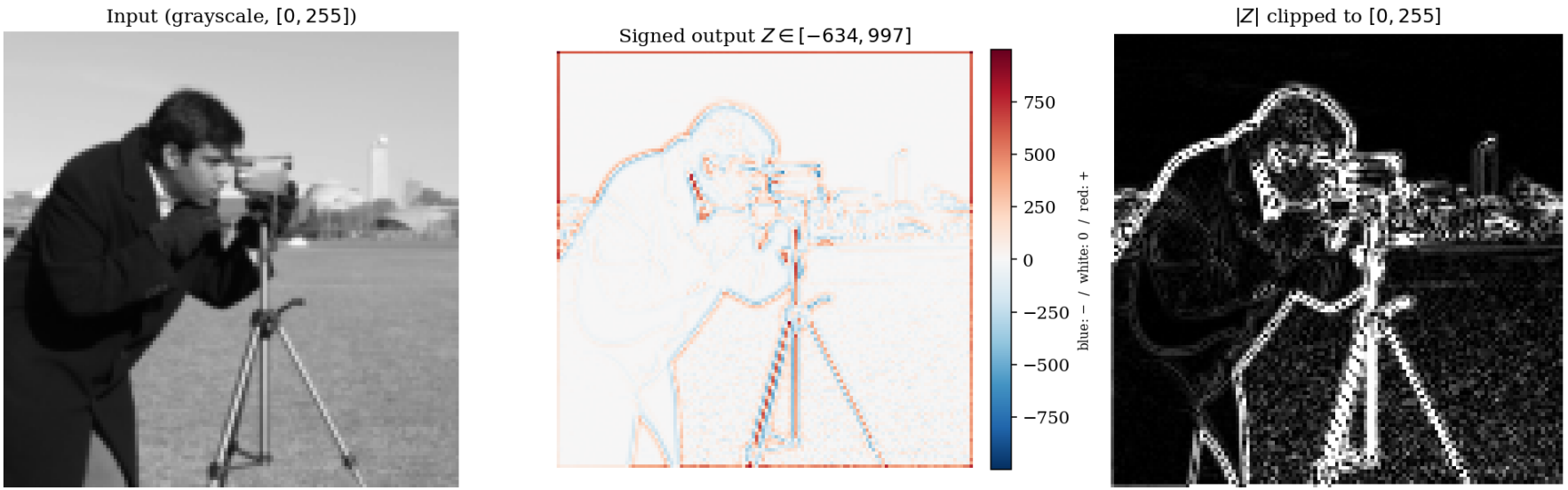
Outline (8-connected Laplacian)

$$k = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}, \qquad z_{ij} = 8x_{ij} - \sum_{\text{8 neighbors}} x$$

Weights sum to zero: flat regions yield $z = 0$. Unlike Sobel, this kernel is *isotropic*: it fires on any change, regardless of orientation.







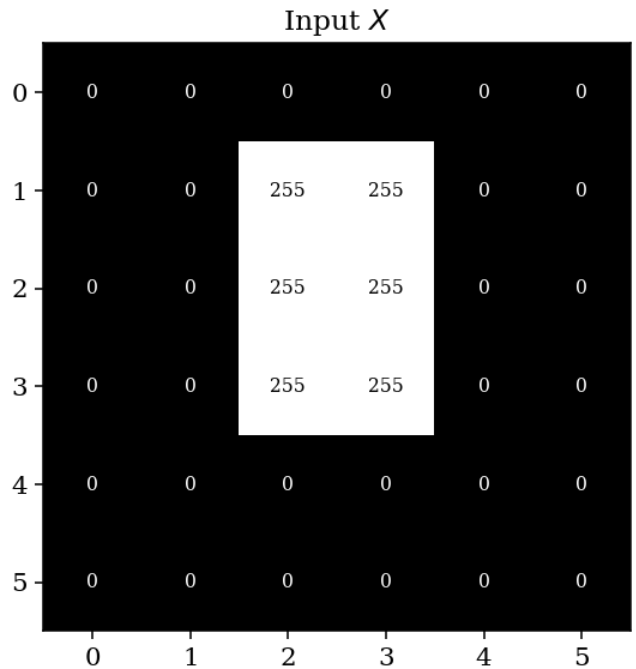
Here, every boundary contributes, regardless of direction (have a look at the flappy part of the jacket).

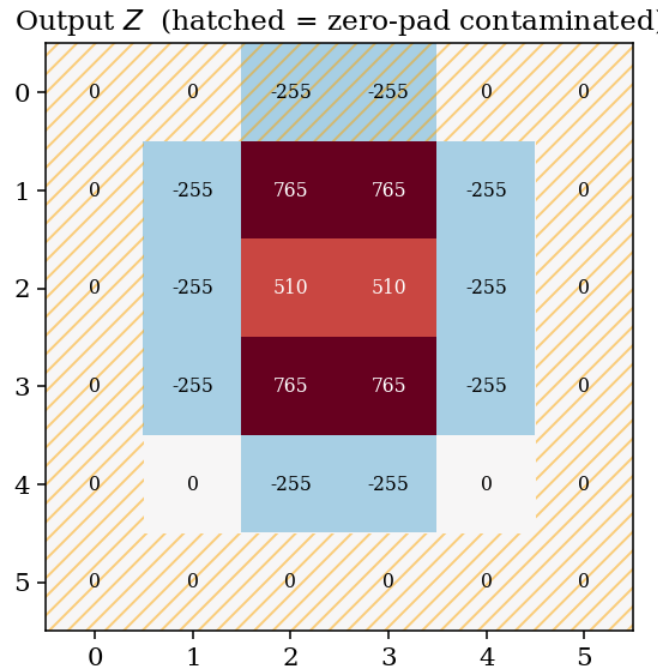
Notice how, for whenever a color gradient happens, we have both a red and a blue line.

Sharpening

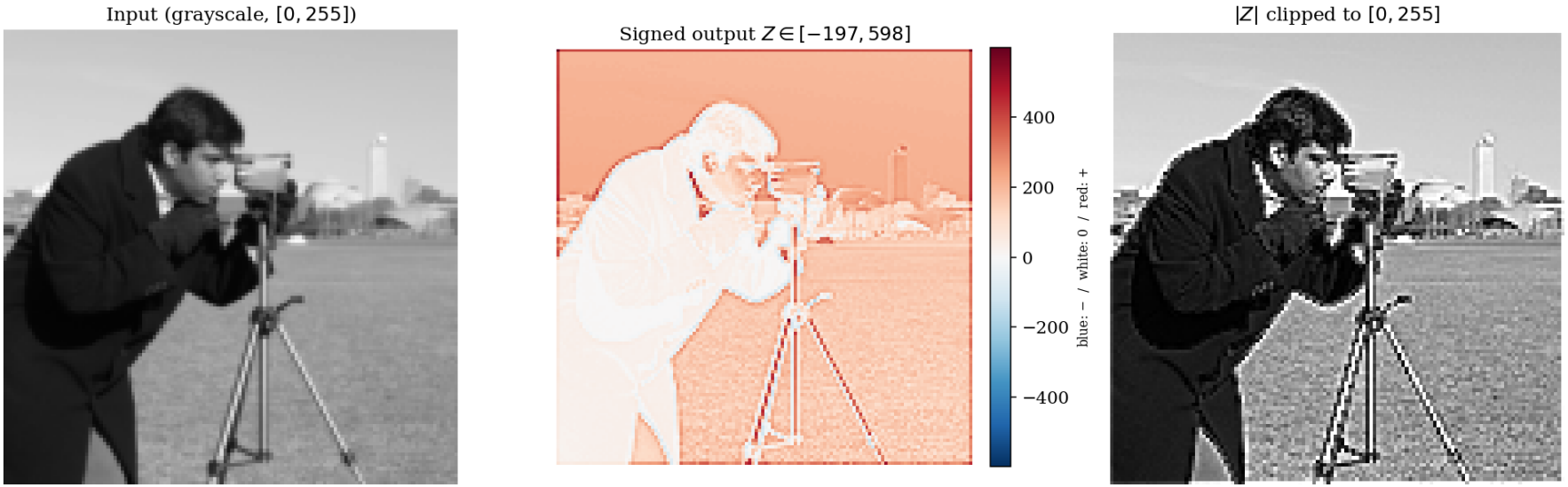
$$k = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}, \qquad z_{ij} = 5x_{ij} - (x_{i\pm1,j} + x_{i,j\pm1})$$

Kernel weights sum to 1. Thus, in flat regions we have $z = x$ and the image is preserved. Near edges, the negative neighbors subtract from the center, producing strong over/undershoots.





Edges become crisper while flat regions stay the same:

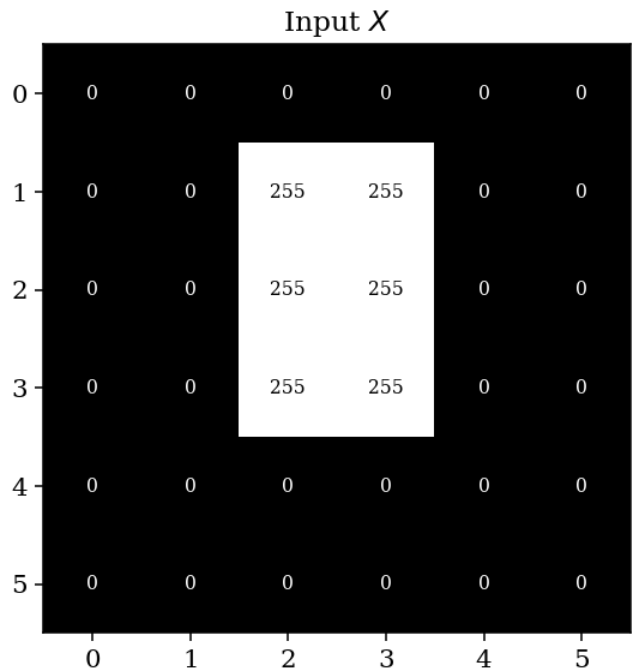


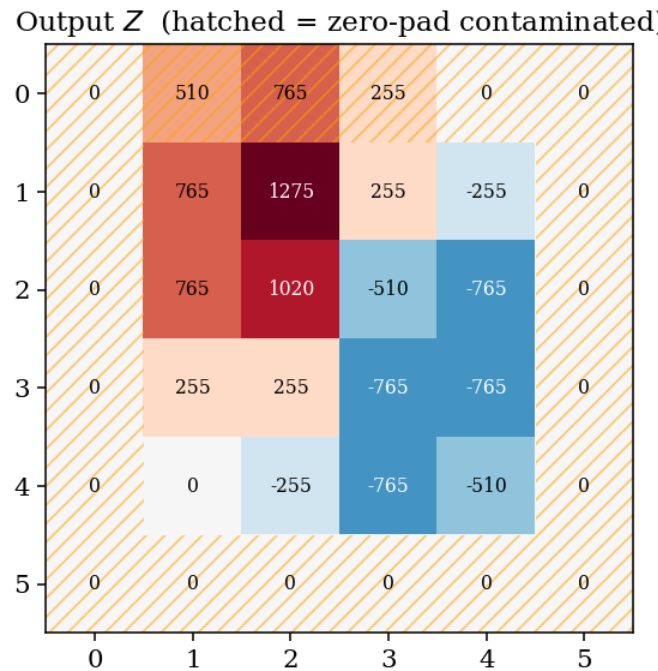
Notice the classical “halo” artifact of over-sharpening near the edges.

Emboss

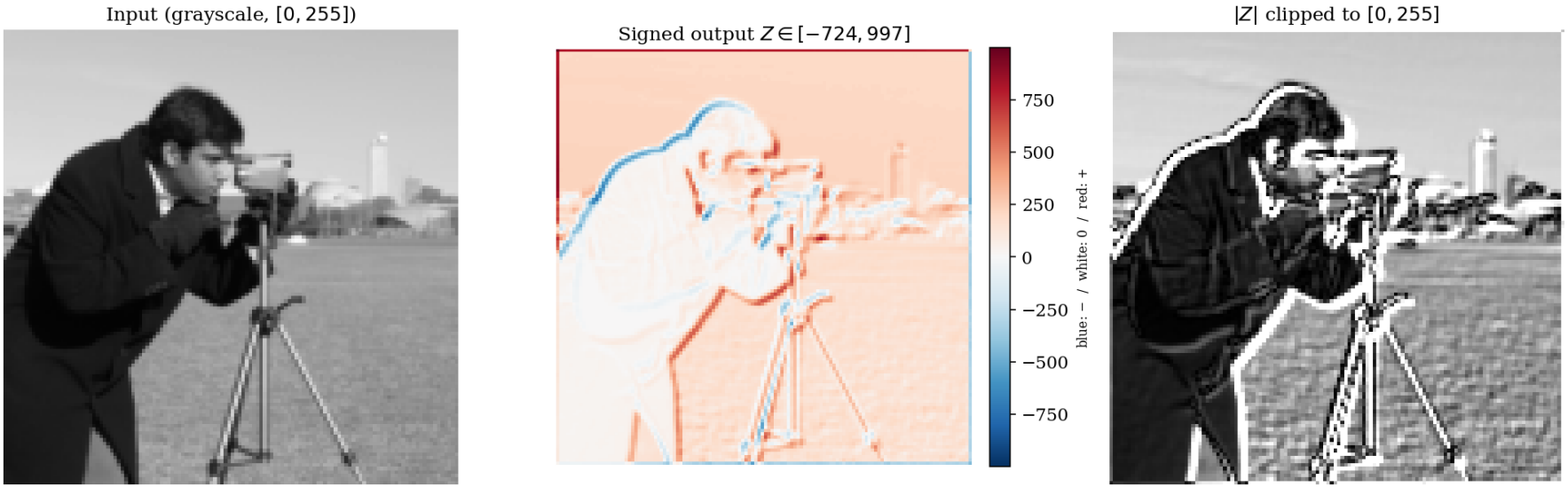
$$k = \begin{pmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix}, \qquad z_{ij} = \sum_{u,v} k_{u,v}\, x_{i+u,\, j+v}$$

Weights sum to 1 but the kernel is *anti-symmetric along the NW–SE diagonal*: it acts like a directional derivative along that axis. NW–SE edges produce strong signed responses; edges perpendicular to it cancel out.





The standard “emboss” filter in image editors.



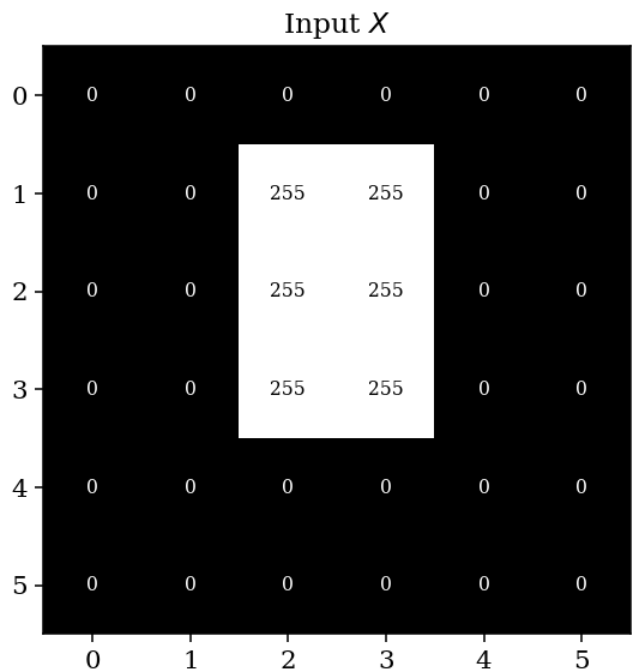
Creates a chiseled, 3D-relief look as if light came from the upper-left: surfaces facing NW look bright, surfaces facing SE look dark.

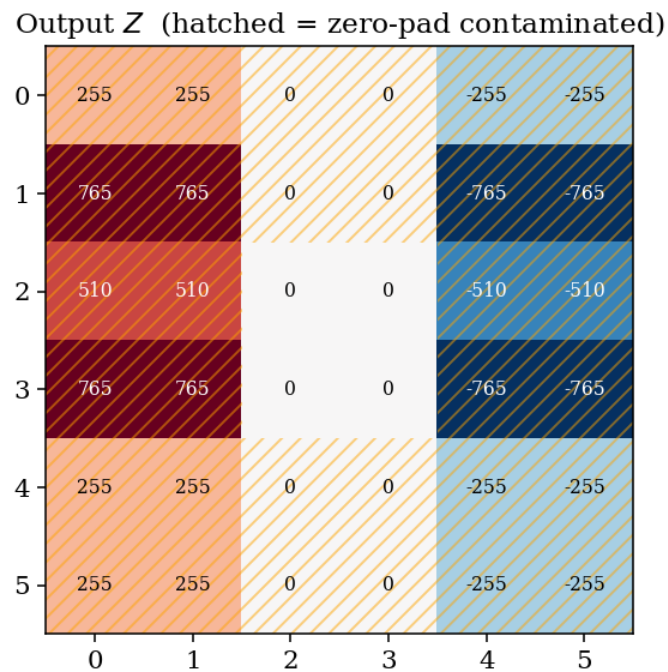
Dilated Sobel-x ($d = 2$)

Same Sobel-x kernel, but taps spaced 2 cells apart:

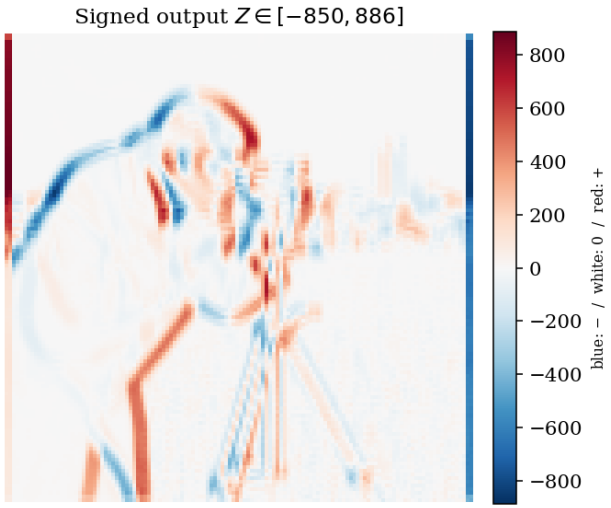
$$z_{ij} = \sum_{u,v \in \{-1,0,1\}} k_{u,v}\, x_{i+2u,\, j+2v}$$

The receptive field grows from 3×3 to 5×5 at *no extra parameters*. Notice how the "edge response shifts outward" due to the kernel reaching further from the center, and the zero-padding boundary widens to a 2-cell rim.



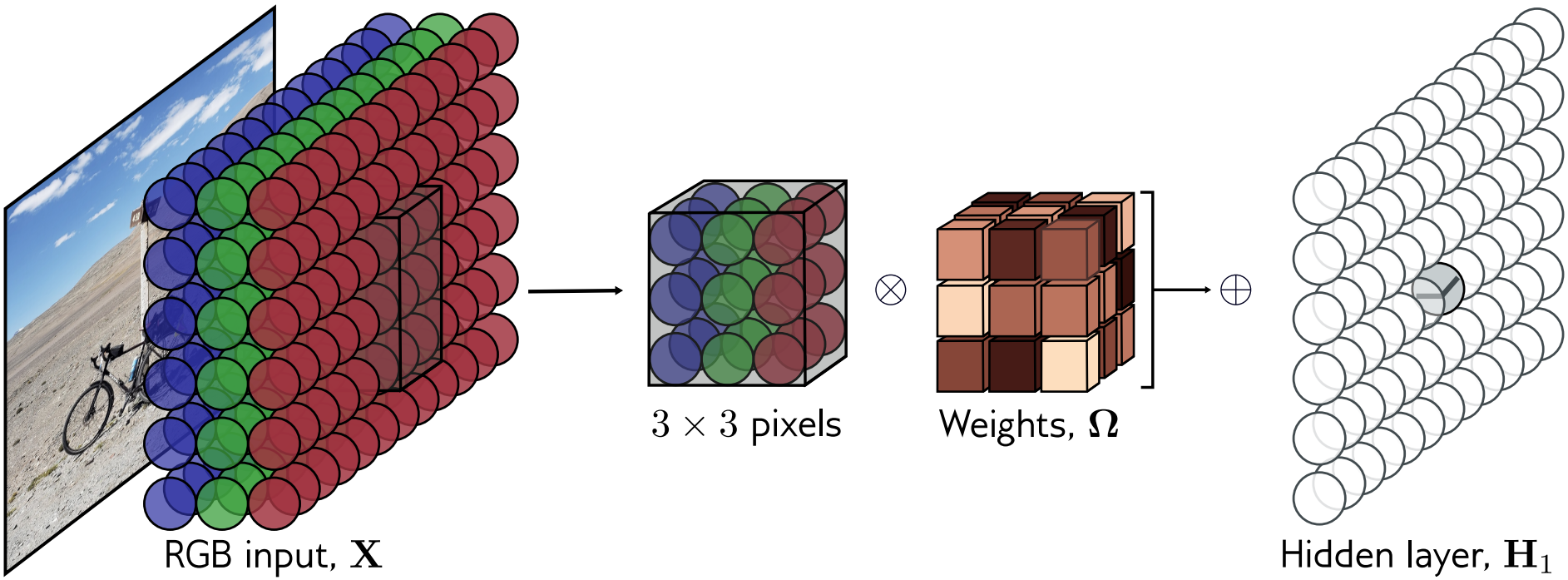


Edges appear visibly *thicker* than in plain Sobel-x because each output pixel integrates over a wider input region.



2D convolution with channels

The kernel is systematically translated across the input in both horizontal and vertical directions, generating an output feature map. If the input is an RGB image, it is treated as a 2D signal with three channels (Red, Green, and Blue). In this case, a 3×3 kernel has $3 \times 3 \times 3$ weights (a **tensor**) and is applied across all three channels at each spatial position, producing a 2D feature map.



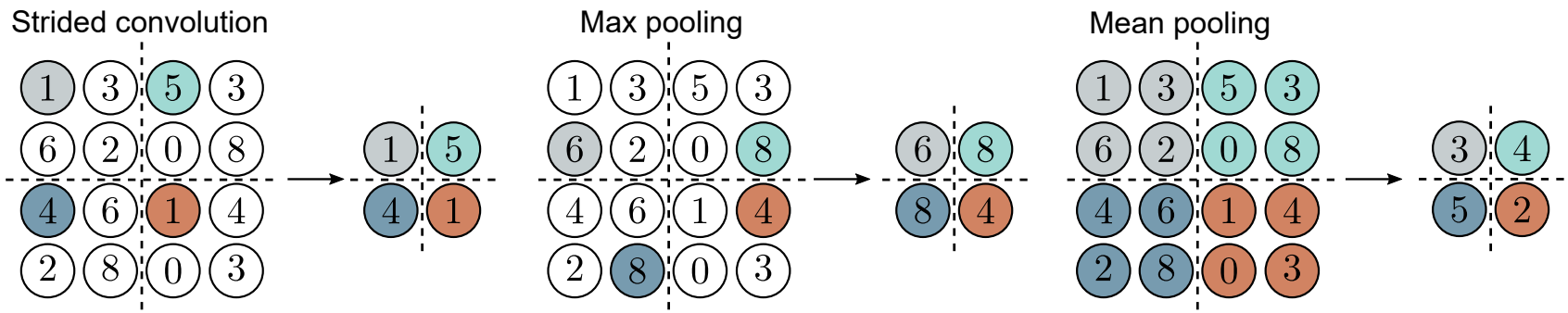
(Prince (2023), MIT Press)

6. Downsampling and 1×1 convolution

Downsampling

Downsampling reduces the resolution, increasing the receptive field and computational efficiency. Three primary methods are used to reduce spatial resolution (each applied independently to each channel).

- **Strided Convolution:** Applying a convolution with a stride of two effectively samples every other position while performing the convolution.
- **Max Pooling:** The maximum value from each 2×2 region is retained, providing some translation invariance.
- **Mean Pooling:** The mean of each 2×2 region is computed, providing a smoother downsampling approach.



(Prince (2023), MIT Press)

Various downsampling methods, such as max pooling (Zhou & Chellappa, 1988) and mean pooling (LeCun et al., 1989), were introduced early on. Comparative studies (Scherer et al., 2010) suggested that max pooling generally performed better.

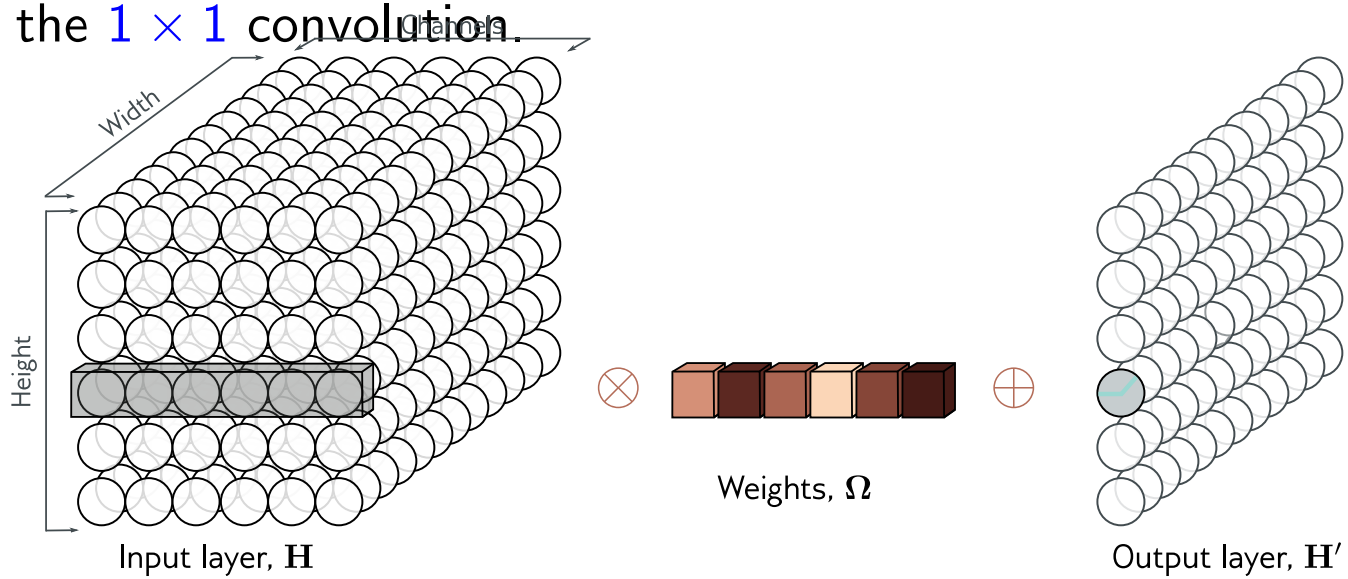
Changing the number of channels

1×1 convolution is used to modify the number of channels between layers without altering spatial resolution.

A 1×1 convolution at each position computes a weighted sum of all input channels:

$$z = \sum_{c=1}^{C_i} \omega_c x_c + b$$

where C_i is the number of input channels, and ω_c are the weights. This is equivalent to applying the same $C_{in} \times C_{out}$ linear transformation independently at every spatial position (i, j), without mixing information across positions—unlike a plain linear layer applied to the whole feature map, which would conflate all pixels and destroy spatial structure. The resulting number of output channels depends on the number of filters in the 1×1 convolution.



(Prince (2023), MIT Press)

7. Training CNNs to achieve invariance

Invariance

The output of a CNN should be **invariant** to different image transformations: a NN used for image classification should recognize an object as the same whether it is translated, rotated, rescaled, or flipped.

(Bishop (2023), Springer)

Learning invariances in neural networks

The locality of the receptive field adopted in CNNs naturally promotes translation invariance.

To enforce the other properties, one typically resorts to **data augmentation**, which involves expanding the dataset by applying transformations to training samples by including rotated, translated, or scaled versions of the training set.

This approach effectively reduces the need for extensive training data (i.e., featuring photos of the same cat in different poses) by artificially augmenting it.

8. Closing

LeNet-5: putting it all together

All the ingredients we have seen (convolution, pooling, channels, fully-connected layers) come together in LeCun's LeNet-5 (1998), the architectural template every CNN since has inherited.

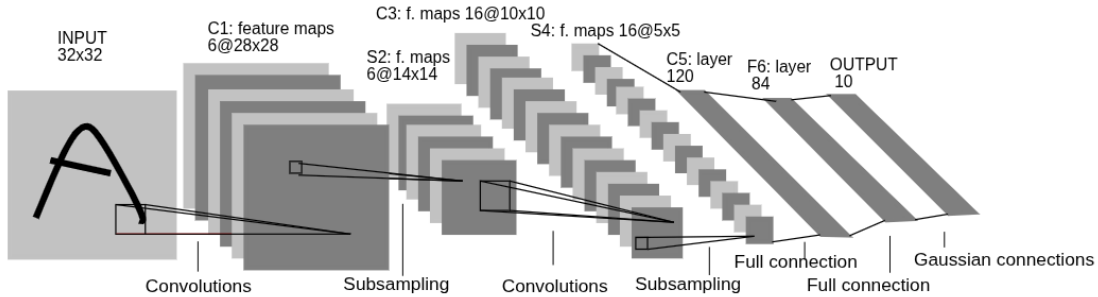


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Layer	Operation	Output shape
Input	32×32 grayscale digit	$32 \times 32 \times 1$
C1	conv 5×5, 6 filters, tanh	$28 \times 28 \times 6$
S2	average pool 2×2	$14 \times 14 \times 6$
C3	conv 5×5, 16 filters, tanh	$10 \times 10 \times 16$
S4	average pool 2×2	$5 \times 5 \times 16$
C5	conv 5×5, 120 filters, tanh	$1 \times 1 \times 120$
F6	fully connected, tanh	84
Output	RBF / Gaussian connections	10 (digit class)

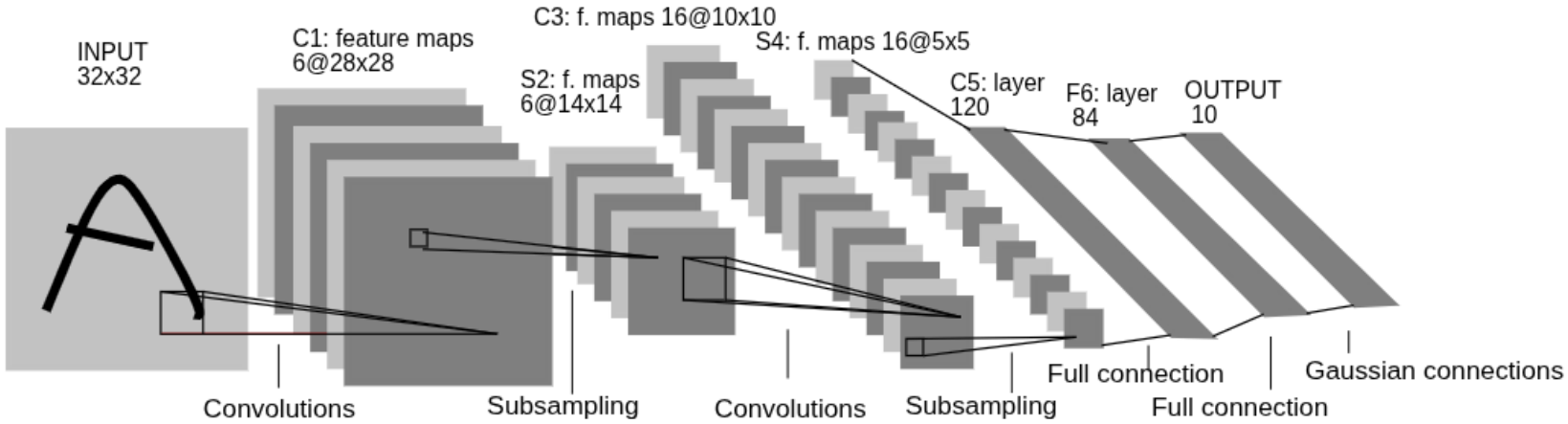


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Notice the signature CNN pattern: **spatial dimension shrinks, channels grow**. The image collapses from a 32×32 grid to a 120-dim vector, ready for the final classifier. About 60k parameters total—compare to modern CNNs with millions or billions.

Note: in LeNet, S2$\to$C3 uses a *sparse* channel-to-channel connection pattern (each C3 filter sees only a subset of S2 channels) to save parameters. Modern CNNs use full connections.

Note: LeNet's original output used *Gaussian/RBF connections* (comparing F6 with a hardcoded bitmap of each digit). Modern networks replace this with the standard fully-connected + softmax + cross-entropy.

From LeNet to modern CNNs

LeNet's recipe survived for fifteen years before being scaled up. The transition to modern CNNs (AlexNet, VGG, ResNet—next lecture) involved *quantitative* growth and a few *qualitative* changes to the recipe.

What changed quantitatively:

- **Depth**: from 7 layers (LeNet) to 8 (AlexNet) to 19 (VGG) to 152 (ResNet) and beyond.
- **Parameters**: from 60k (LeNet) to 60M (AlexNet) to 138M (VGG). Modern foundation vision models cross the billion mark.
- **Data**: from 60k MNIST digits to 1.2M ImageNet images. Modern models train on billions.
- **Compute**: from a CPU at LeCun's desk to thousands of GPUs.

What changed qualitatively:

- tanh $\rightarrow$ **ReLU** (Krizhevsky, 2012): trains deep networks without vanishing gradients.
- Average pool $\rightarrow$ **max pool**: more invariance, simpler gradients.
- Plain stacking $\rightarrow$ **residual connections** (He, 2015): enables networks of arbitrary depth.

Closing thoughts

Three take-away ideas:

- **Inductive bias is power**. CNNs work better than MLPs on images not because they have more parameters—they have far fewer—but because their architecture matches the structure of the data: locality, hierarchy, translation equivariance. The bias is wired in, not learned.
- **The brain was a useful prior**. Hubel and Wiesel's V1 $\rightarrow$ V2 $\rightarrow$ IT cascade is, structurally, what modern CNNs do. We don't claim CNNs are how the brain works—but the brain was a generative metaphor that pointed in the right direction.
- **Tacit knowledge in weights**. Just like a child who learns to distinguish cats from dogs without articulating a rule (Polanyi, Reber), a CNN compresses millions of examples into a few million numbers and can then generalize. The "rules" are real—they are just no longer human-readable.